Stacklyst

Laboratório de Engenharia de Software



Processos de negócio, modelagem e funcionamento

Pedro Antonio Silvestre Tescaro
Guilherme Alves de Campos
Paulo Henrique Leal dos Santos

Seminário do projeto

O projeto e o problema

Estudo de programação e oportunidades profissionais

Necessidade

Organizar o estudo, praticar com retorno e acompanhar a própria evolução.

Apoio do sistema

Trilhas e lições conectam conteúdo, exercícios e progresso do estudante.

Percurso apresentado

Após a aprendizagem, o grupo publica uma oportunidade, registra a candidatura e acompanha a seleção.

Ordem da apresentação

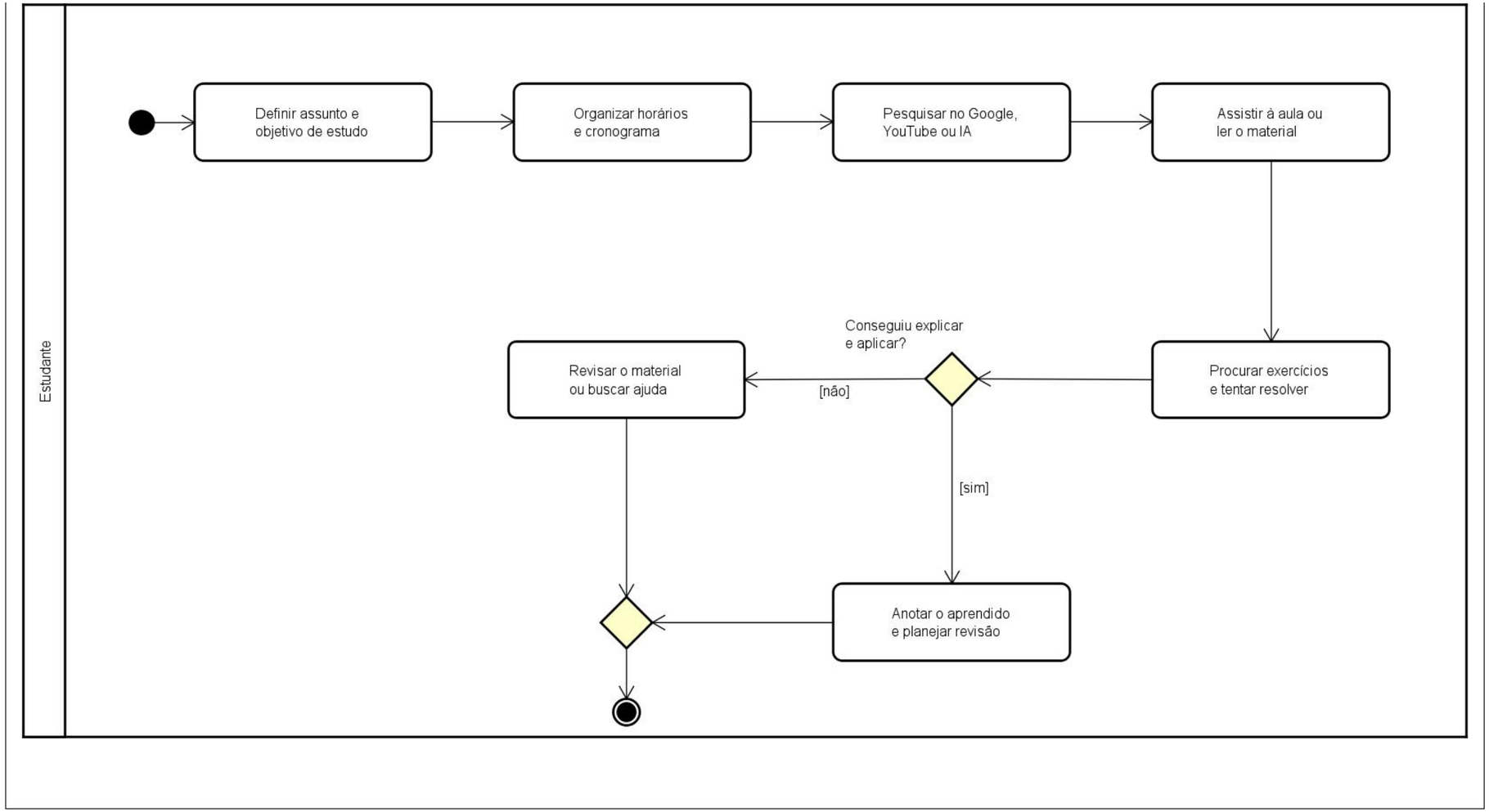
A continuidade entre os integrantes orienta a demonstração

Responsável	Funcionalidade e continuidade	Tempo
Grupo	Projeto, atividades do negócio e casos de uso gerais	9 min
Pedro	Lição e exercício, modelos gerais e progresso	13 min
Guilherme	Publicar a vaga que será usada na candidatura	12 min
Paulo	Candidatura, avanço da seleção e notificação	14 min
Grupo	Encerramento e perguntas	7 min

Planejamento de 55 minutos, com 5 minutos de margem até o limite de uma hora.

Estudar programação

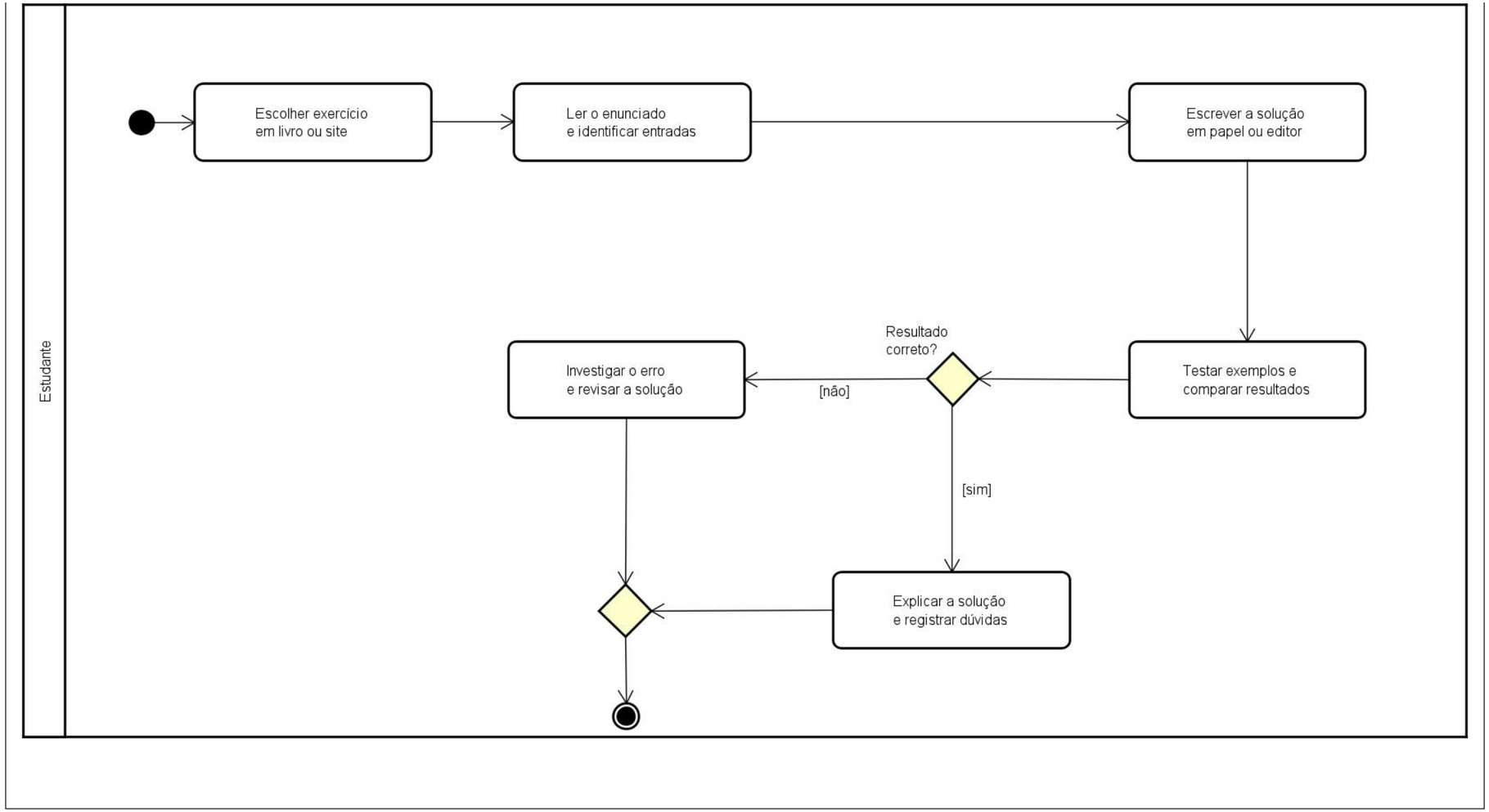
AN01 Atividades do negócio



Organizar o estudo e verificar se o conteúdo foi compreendido e aplicado.

Praticar exercícios de programação

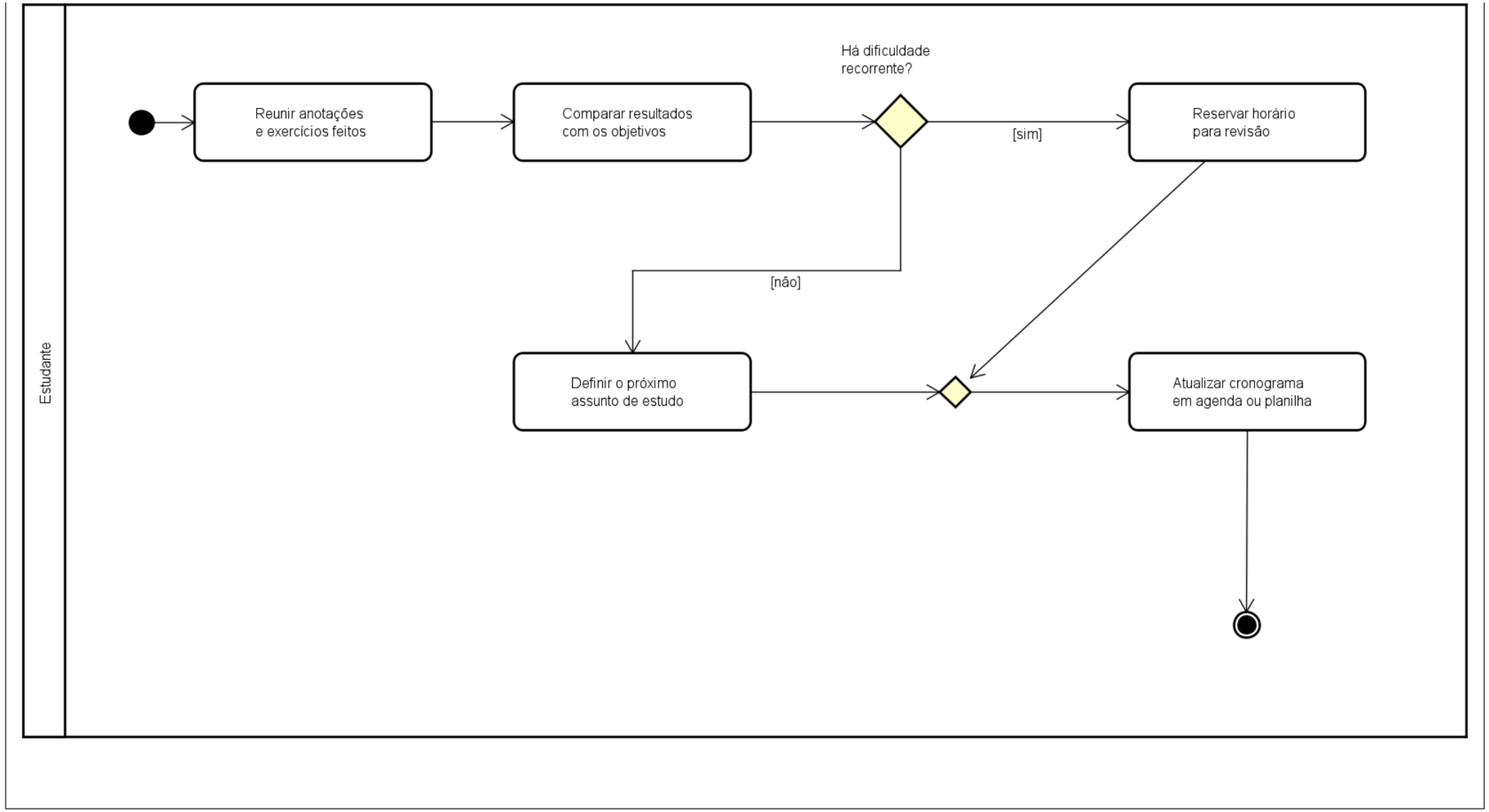
AN02 Atividades do negócio



Resolver um problema, testar exemplos e compreender os erros encontrados.

Acompanhar a própria aprendizagem

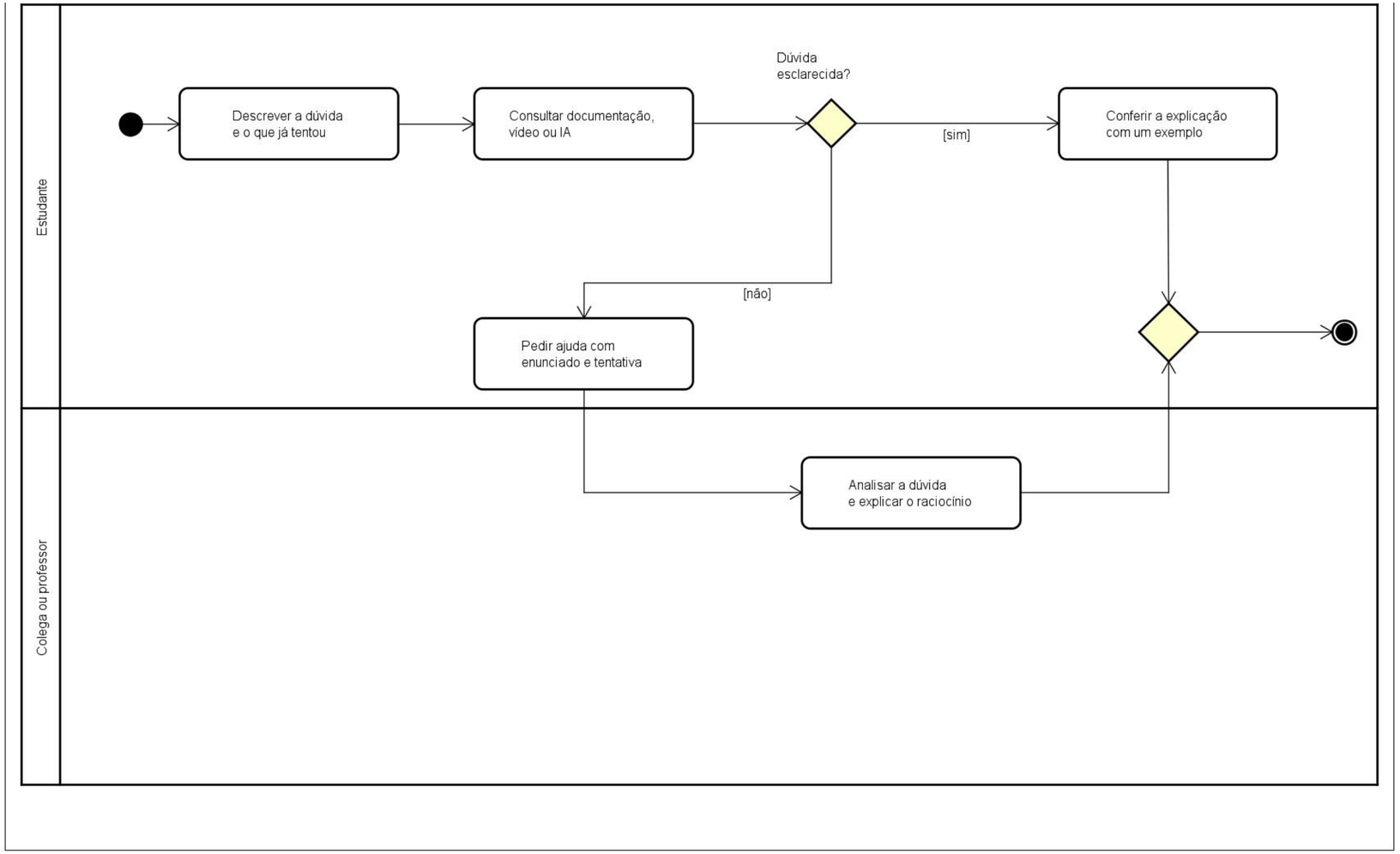
AN03 Atividades do negócio



Comparar os resultados aos objetivos e ajustar o planejamento de estudo.

Buscar ajuda para uma dúvida

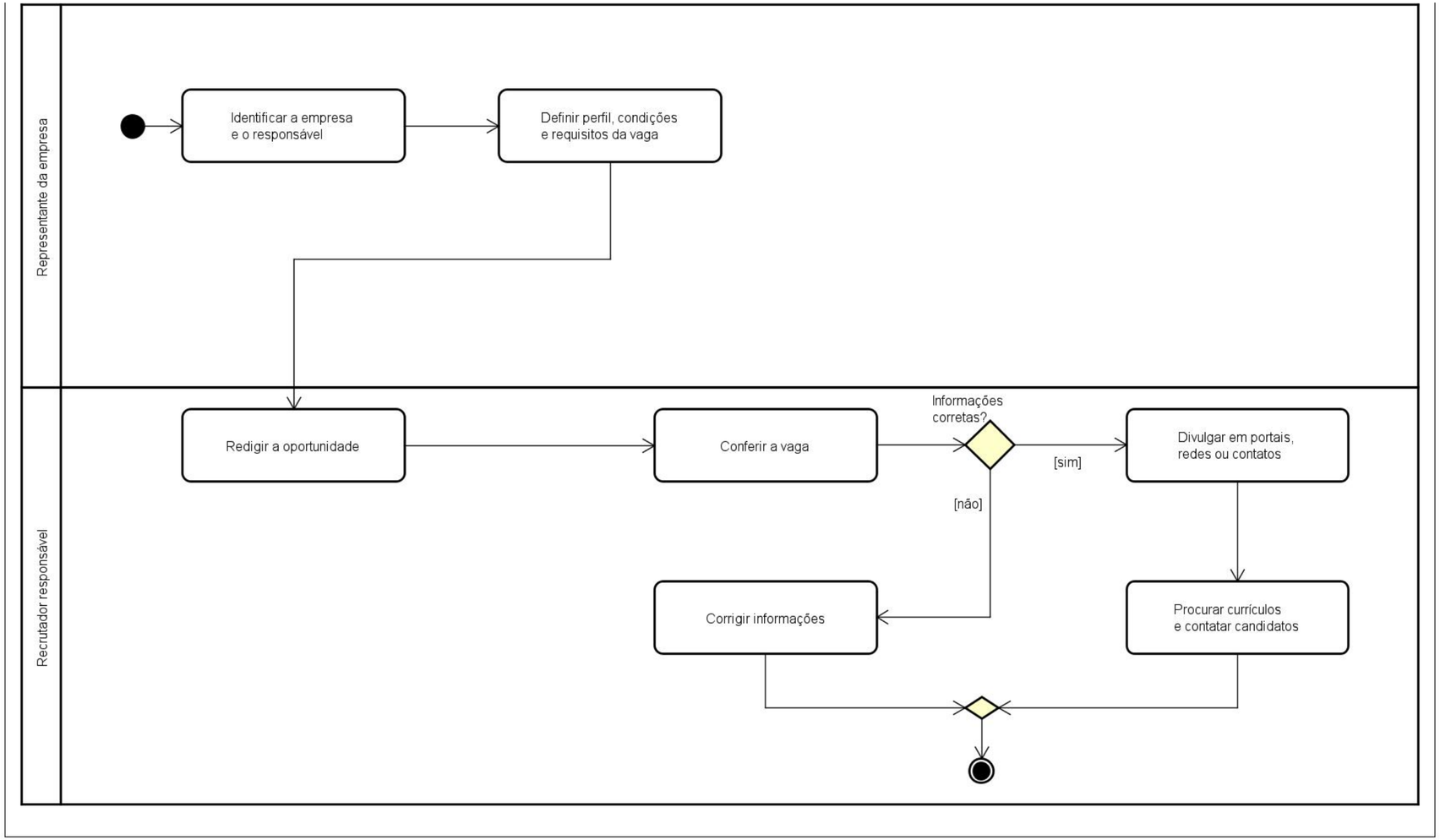
AN04 Atividades do negócio



Esclarecer uma dúvida e conferir a explicação em um exemplo.

Divulgar oportunidade de trabalho

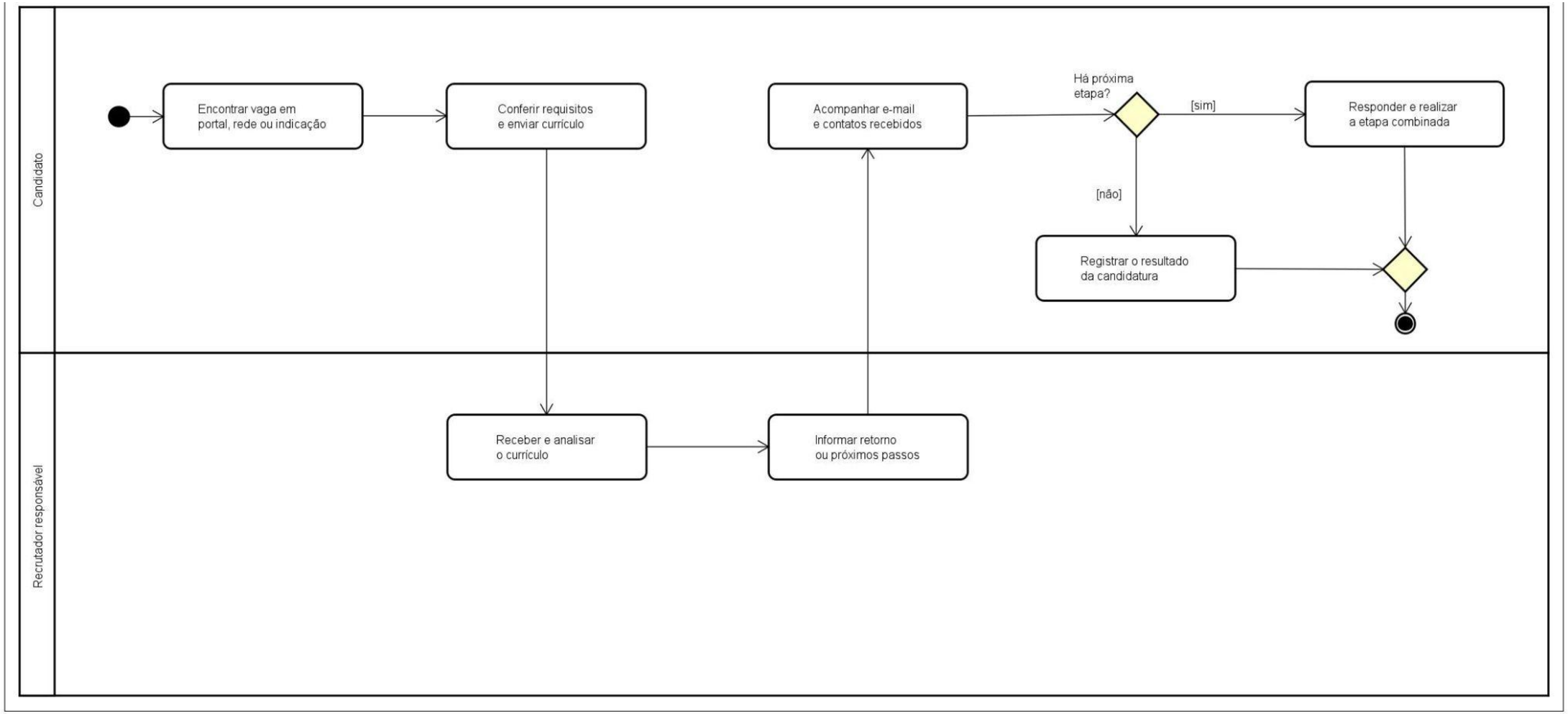
AN05A Atividades do negócio



Definir e divulgar uma oportunidade profissional com informações conferidas.

Candidatar-se e acompanhar a seleção

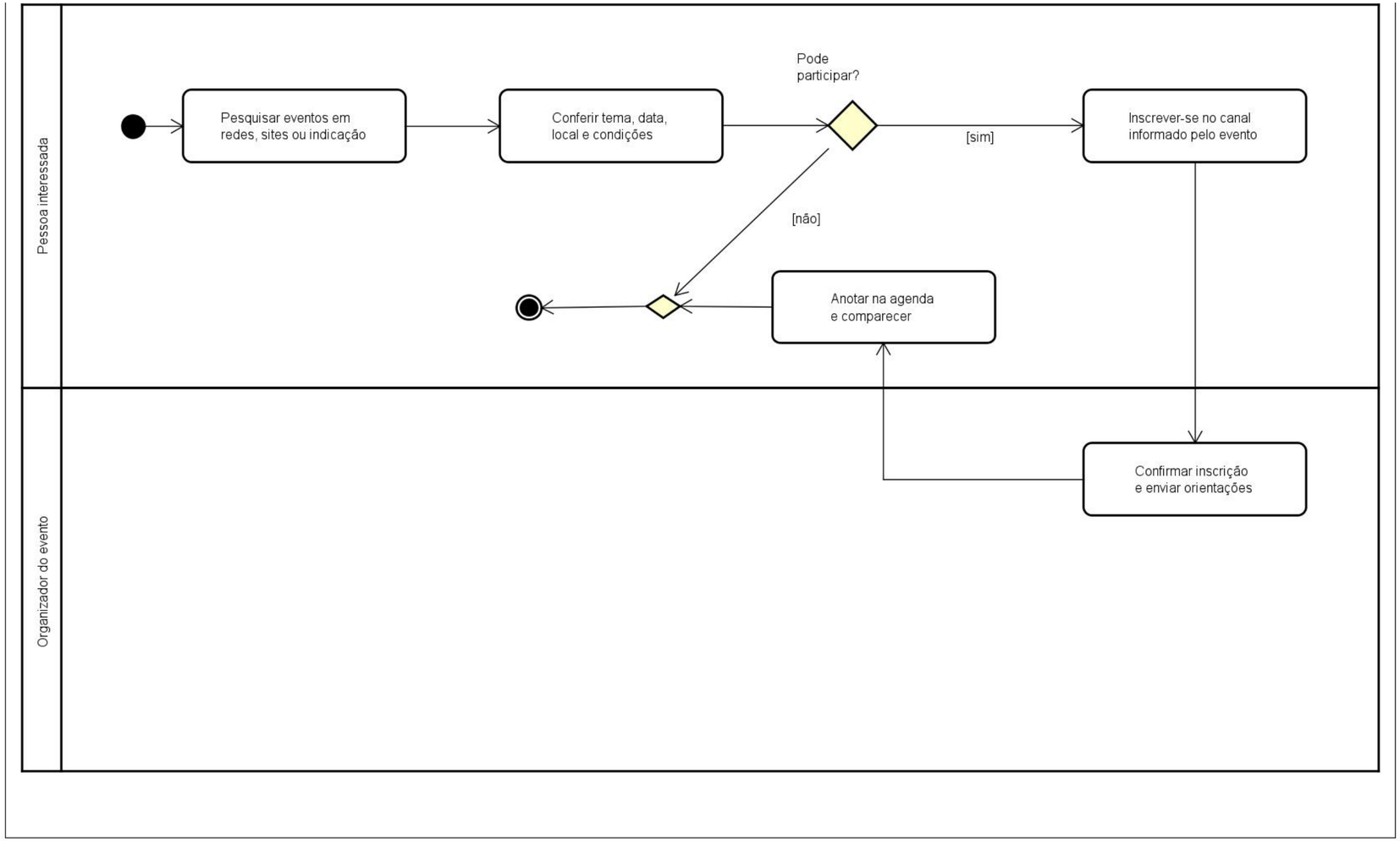
AN05B Atividades do negócio



Enviar o currículo e acompanhar as etapas e o resultado da seleção.

Descobrir e participar de um evento

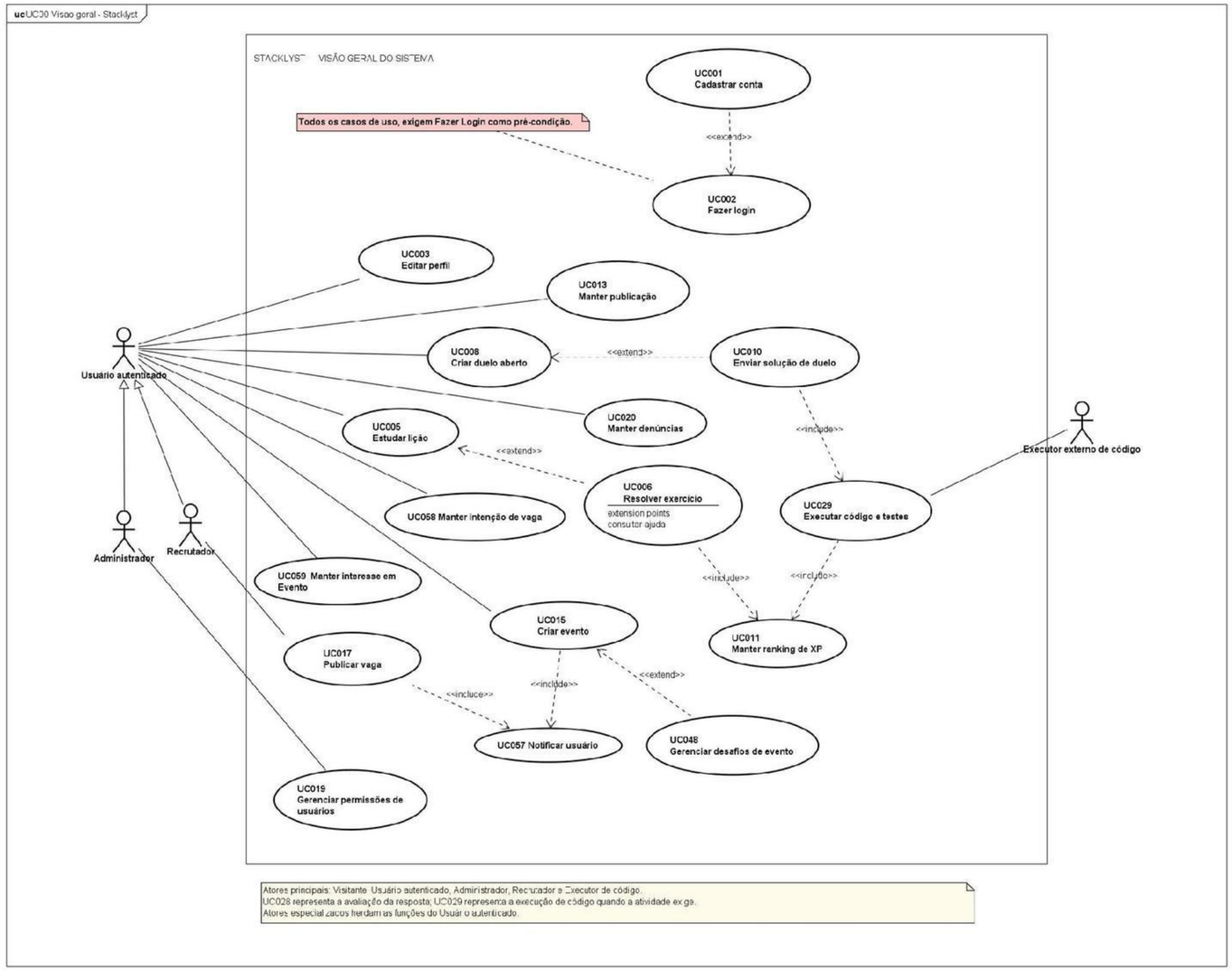
AN06 Atividades do negócio



Encontrar um evento e organizar a inscrição e a participação.

Diagrama geral de casos de uso

18 elementos na imagem geral do documento 04



Visão completa da fonte

A imagem está integralmente presente. Os dois slides seguintes ampliam seus elementos.

Correspondência atual

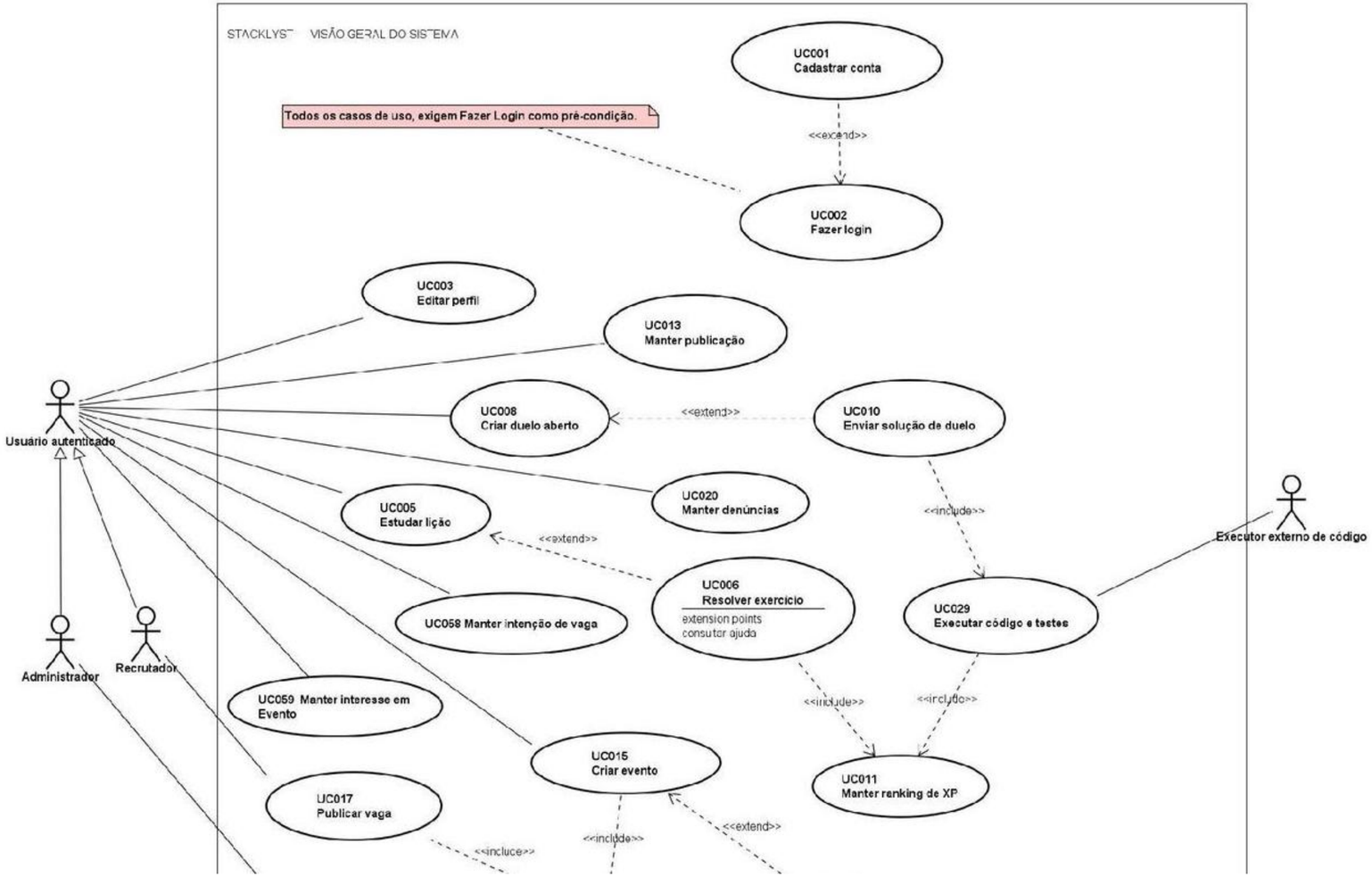
UC022: candidatura à vaga.
UC023: participação em evento.
UC026: atualização e aviso ao candidato.

Leitura dos rótulos

UC057–UC059 são rótulos históricos da imagem. UC011 representa consulta ao ranking.

Casos de uso gerais: acesso e aprendizagem

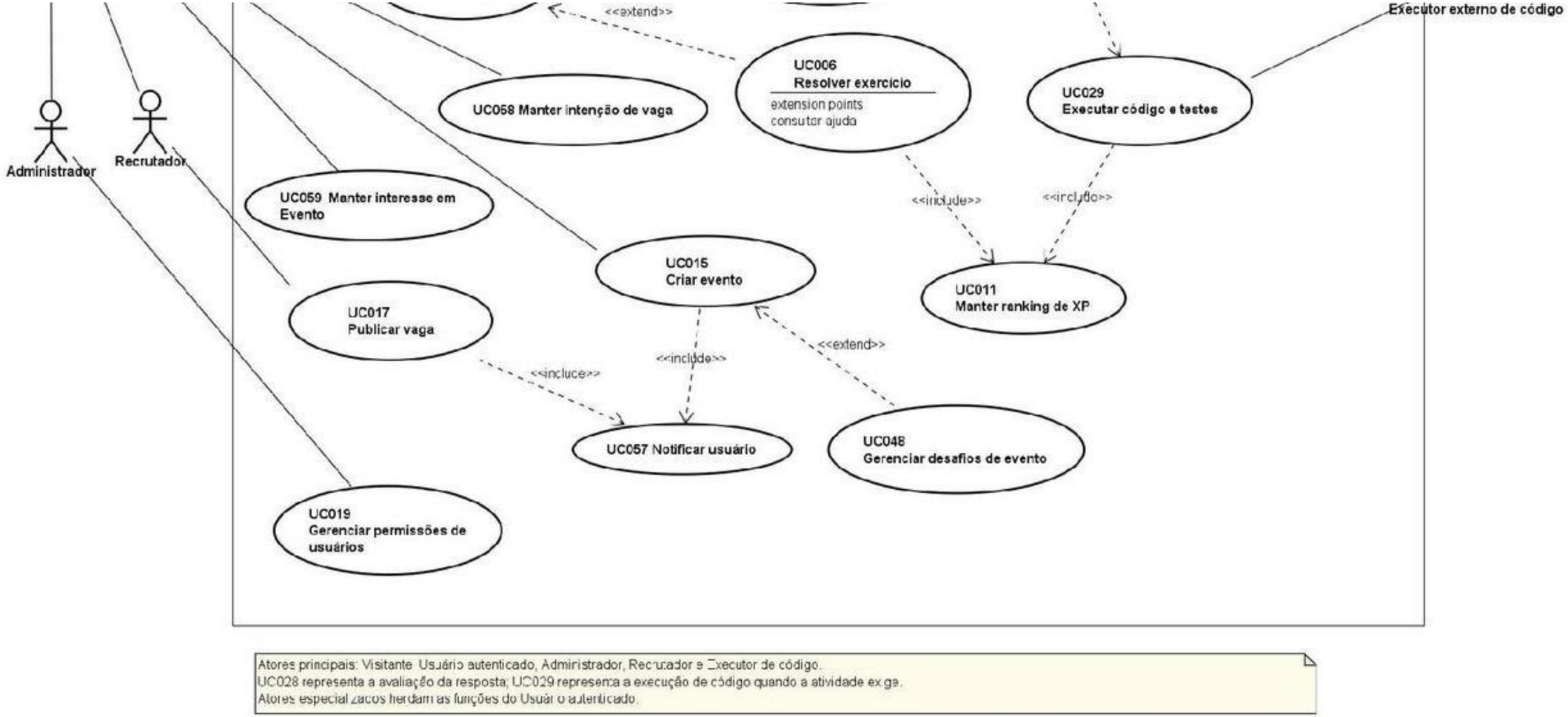
Ampliação da imagem do documento 04



Cadastro e login são públicos. A autenticação é pré-condição dos fluxos protegidos.

Casos de uso gerais: oportunidades e eventos

Ampliação da imagem do documento 04



Na especificação atual: UC022 candidatura, UC023 participação e UC026 atualização com aviso.

Pedro: estudo de uma lição

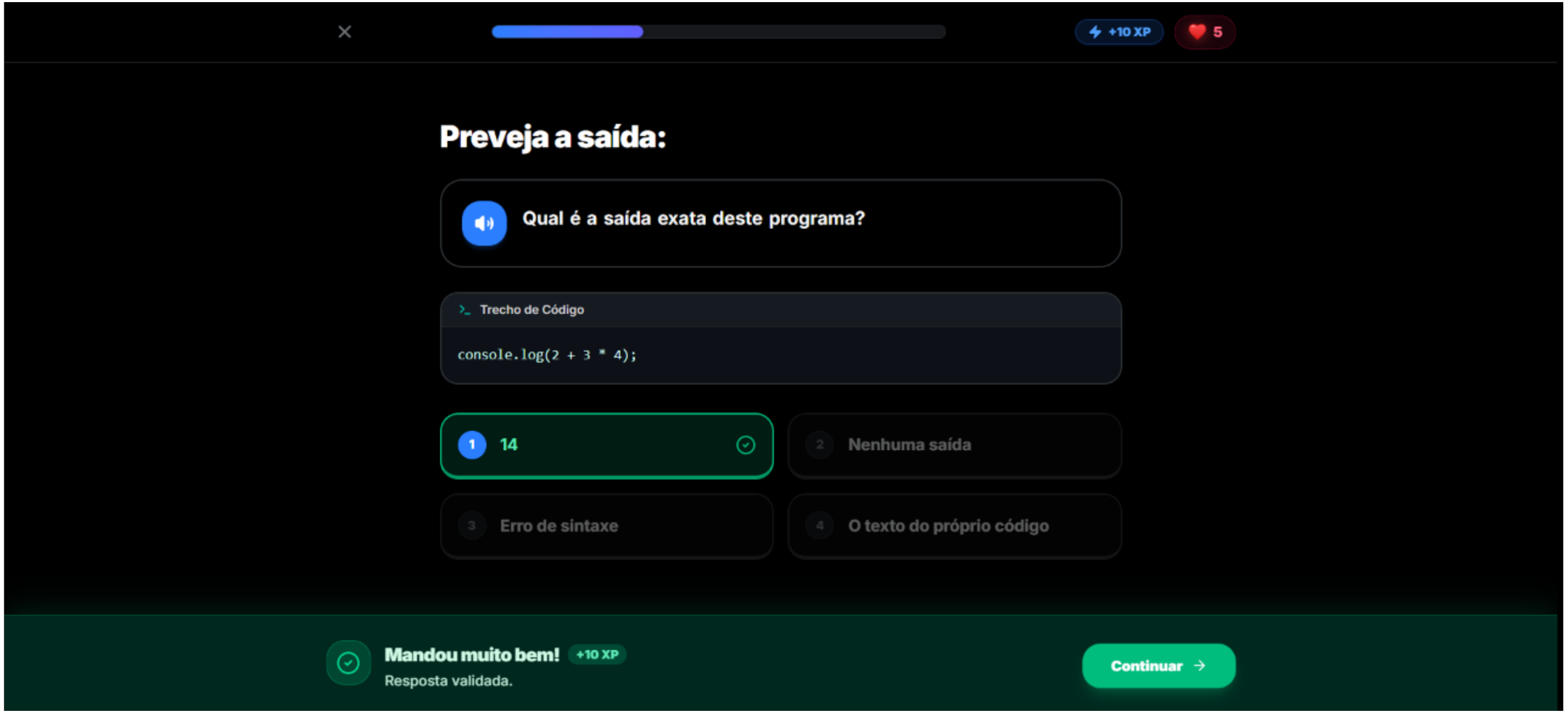
Pedro Antonio Silvestre Tescaro UC005



No protótipo: conteúdo conceitual e navegação entre as etapas da lição.

Resposta e confirmação da atividade

Pedro Antonio Silvestre Tescaro UC006



No protótipo: enunciado, alternativas, validação da resposta e retorno de XP.

Requisitos e regras na interface

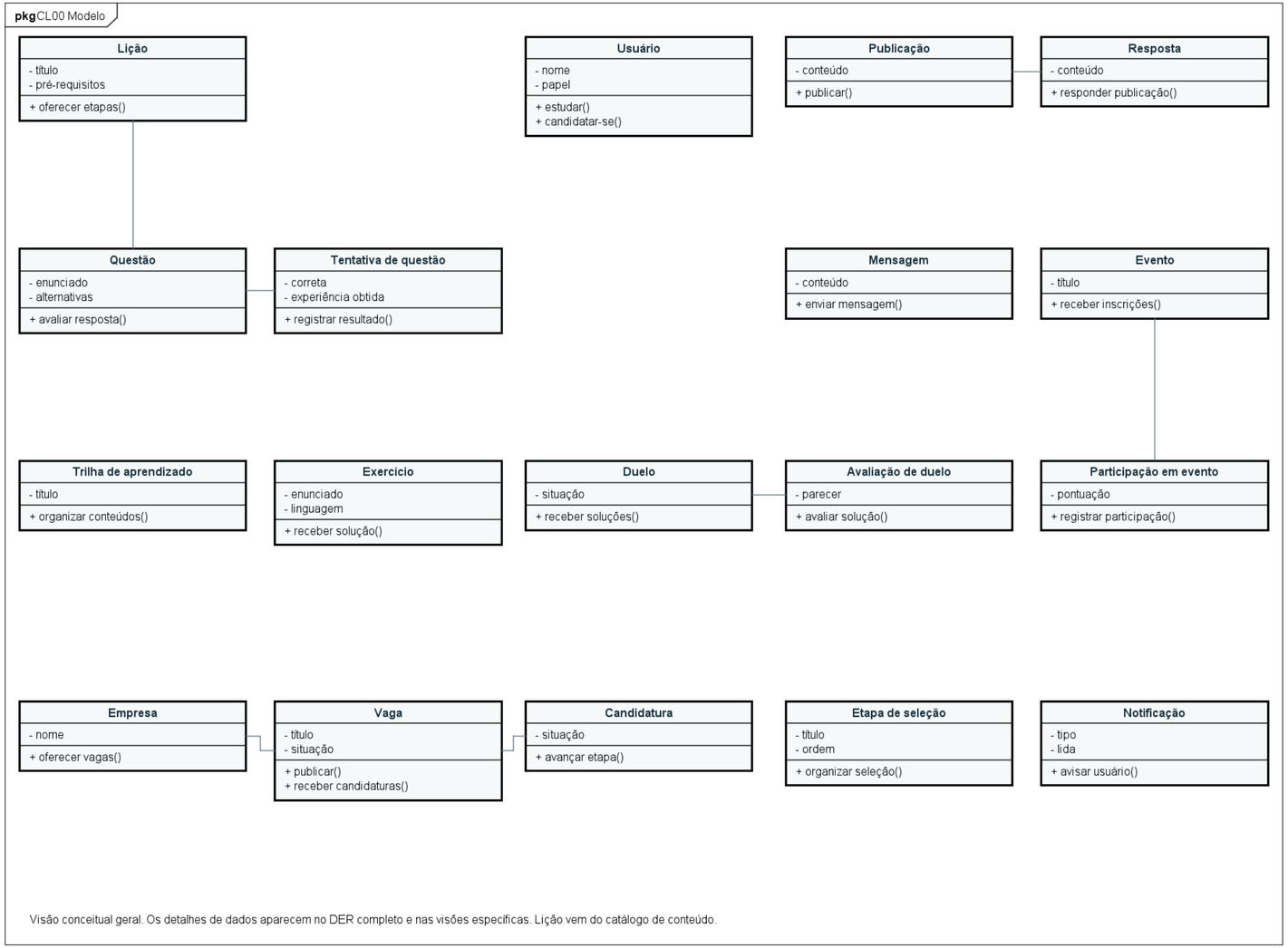
UC005 e UC006 Estudar uma lição e resolver exercício

Requisitos funcionais	Requisitos não funcionais	Regras de negócio
RF007 Acessar lição Exibir a lição e suas etapas interativas. RF008 Resolver exercício Avaliar a resposta no servidor. Testar código não conclui a atividade nem concede XP. RF009 Registrar progresso Gravar tentativa e progresso, sem duplicar a primeira recompensa.	RNF009 Execução limitada Limitar o tempo de execução e o tamanho do código recebido. RNF014 Consistência de XP Atualizar XP e progresso atomicamente, sem duplicidade.	RN001 Elegibilidade de XP Conceder XP somente após atividade válida e confirmada, uma vez. RN029 Pré-requisitos de aprendizagem Exigir os pré-requisitos da lição. Revisitar conteúdo não duplica a recompensa.

RF: o que a tela permite fazer. RNF: qualidade e restrições. RN: condições do domínio.

Diagrama de classes geral

Visão conceitual dos domínios



Modelo de entidades e relacionamentos geral

MER Visão conceitual

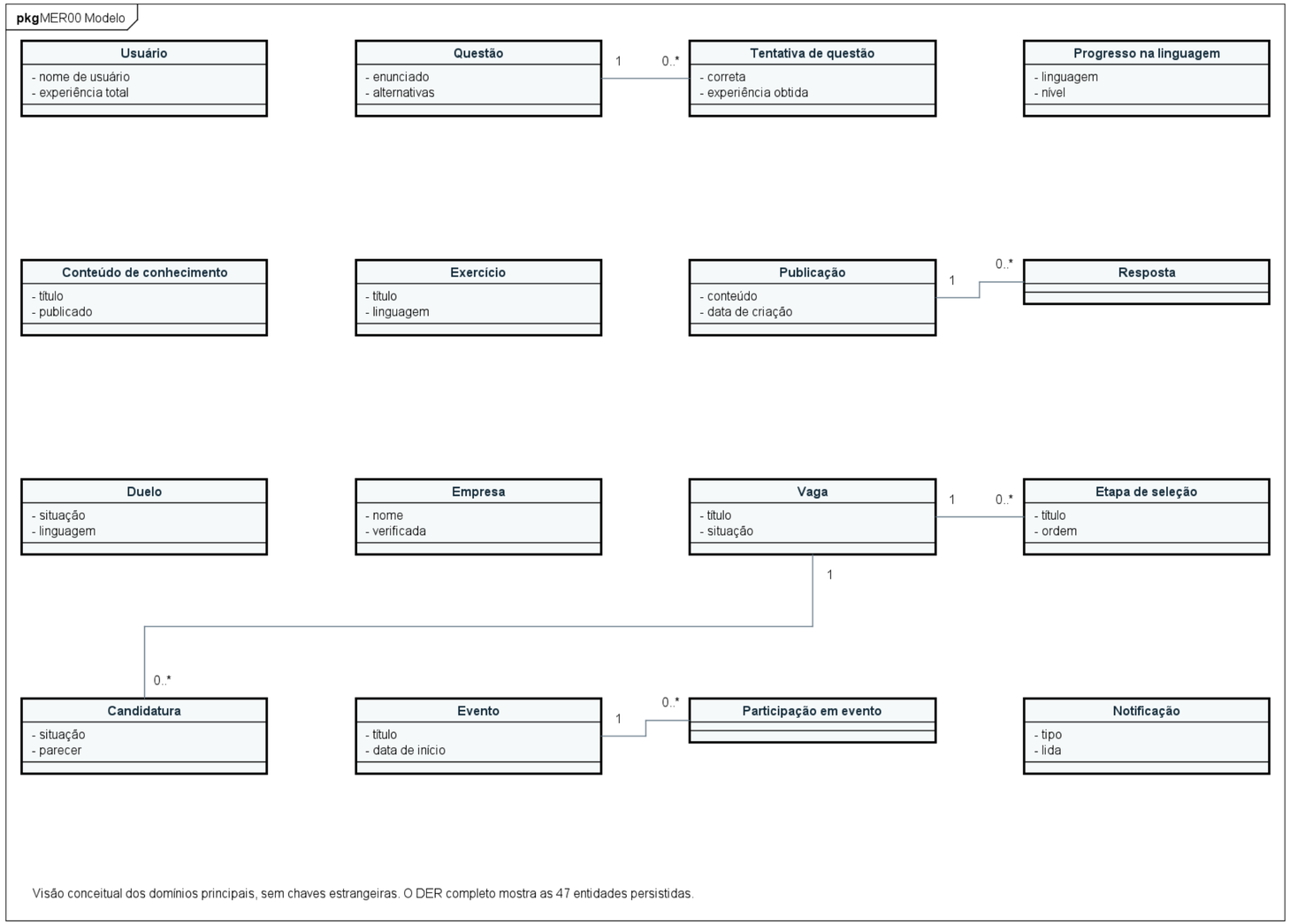
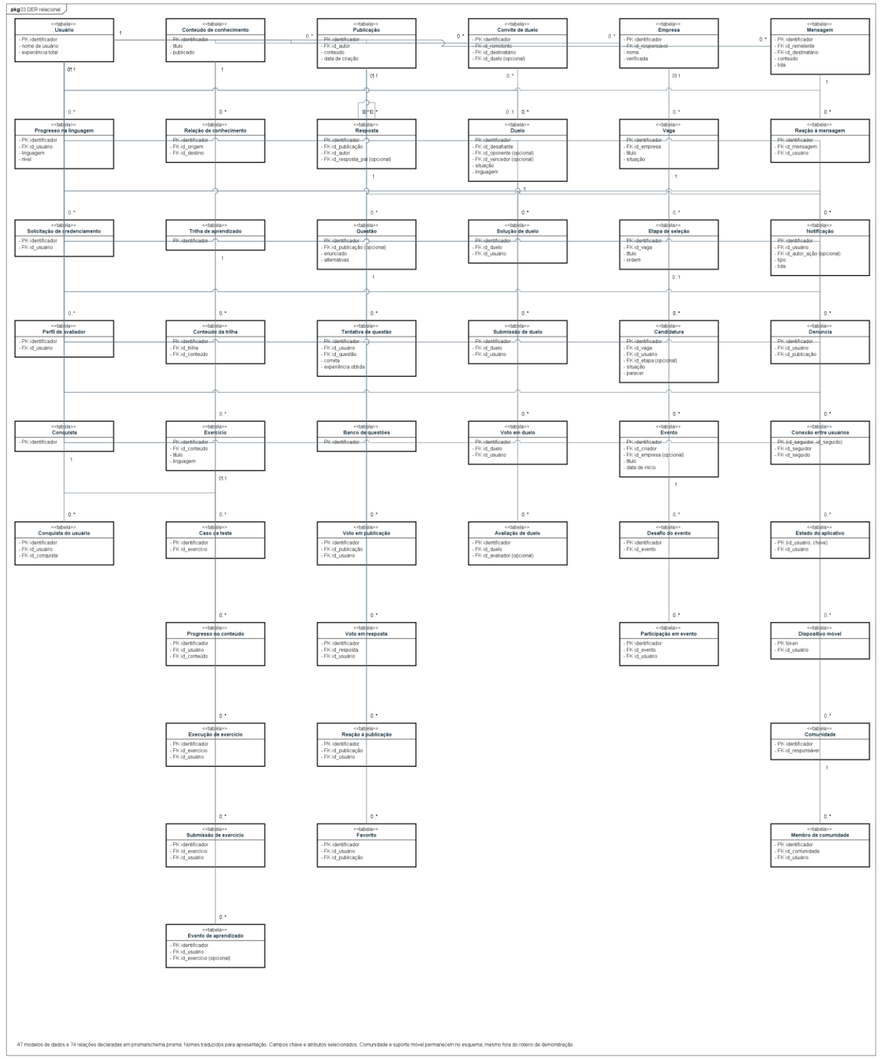


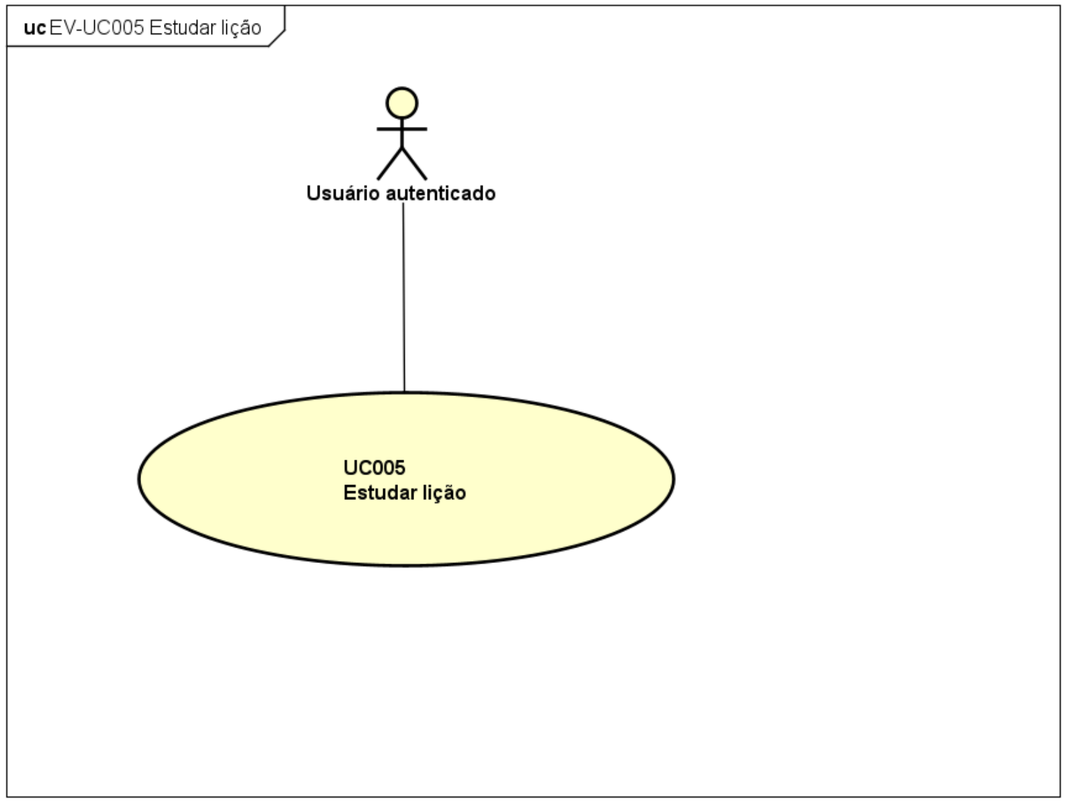
Diagrama geral do banco de dados

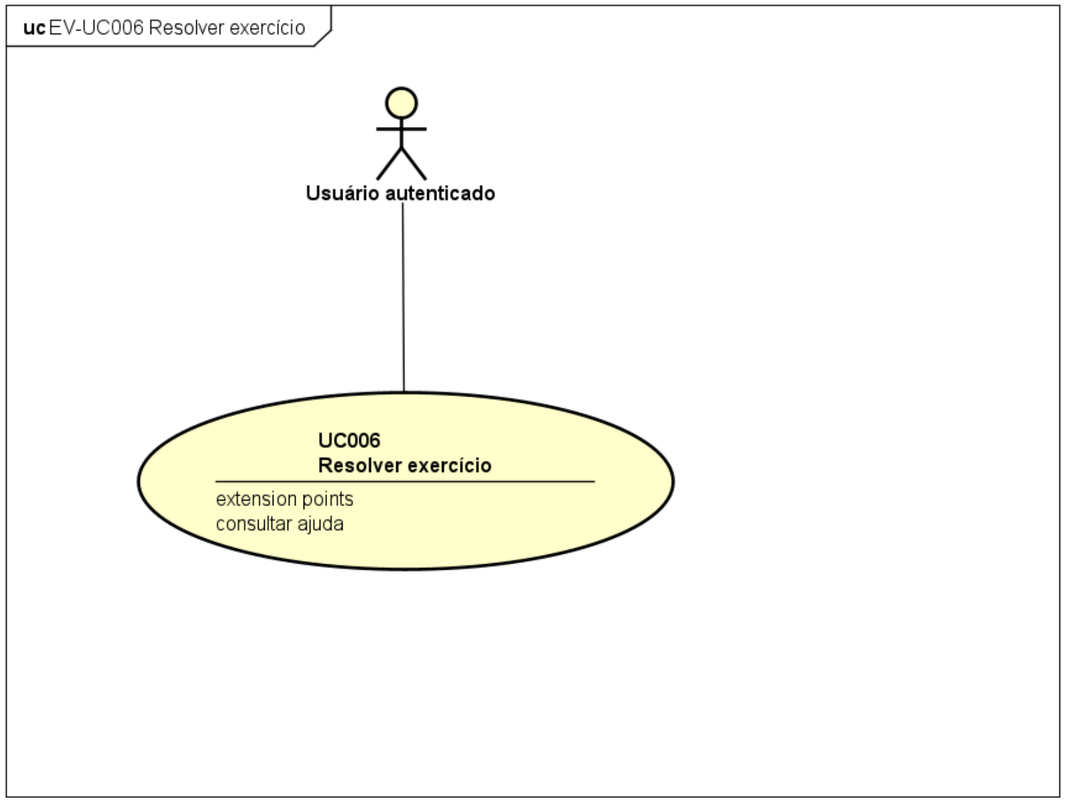
DER Estrutura completa e chaves



Casos de uso da funcionalidade

UC005 e UC006 Estudar uma lição e resolver exercício

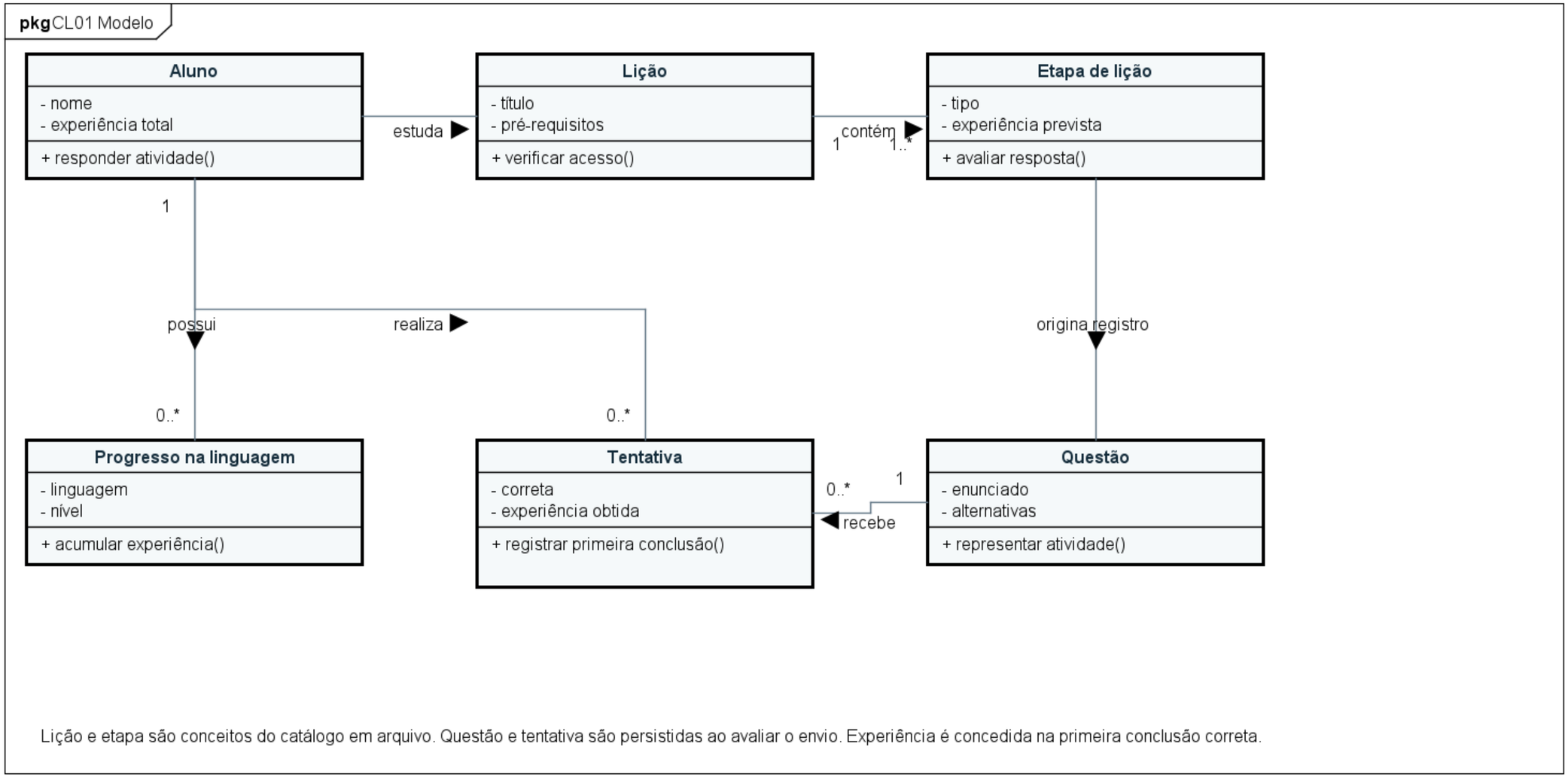




Sessão autenticada como pré-condição dos fluxos apresentados.

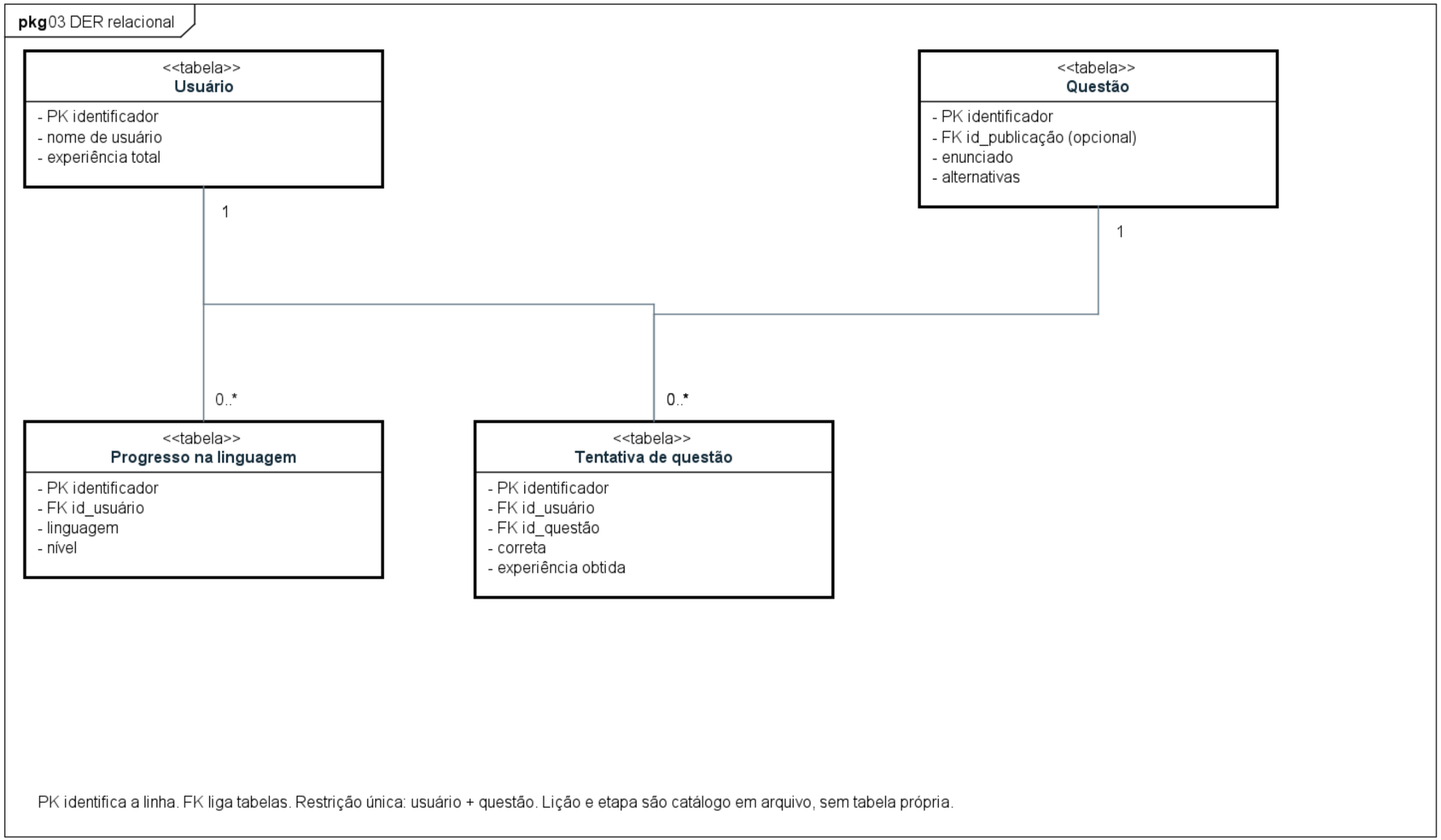
Classes da aprendizagem

UC005 e UC006 Conceitos e responsabilidades



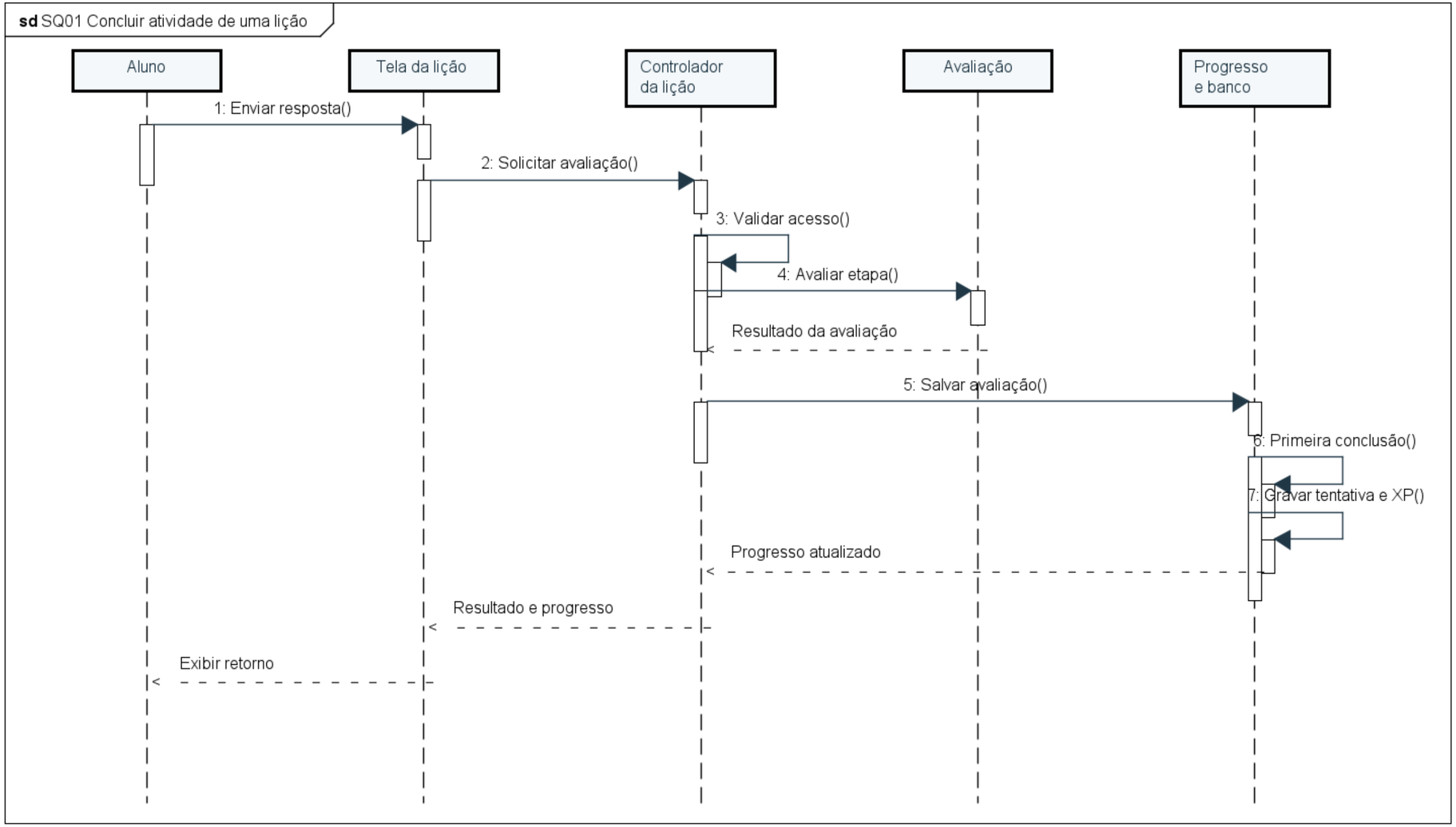
Banco de dados da aprendizagem

UC005 e UC006 Persistência e progresso



Sequência completa: resolver exercício

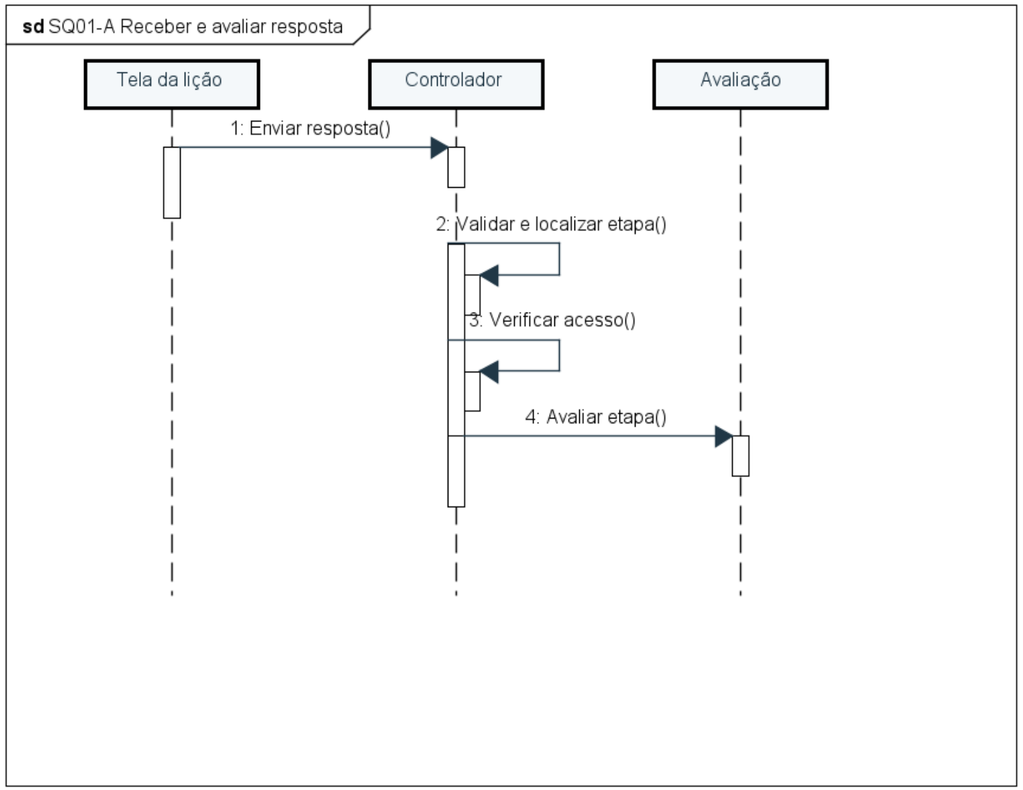
UC006 Fluxo principal



Receber e avaliar a resposta

SQ01-A Recorte da sequência e trecho correspondente

Interação



Implementação

src/app/api/lessons/[lessonId]/attempt/route.ts

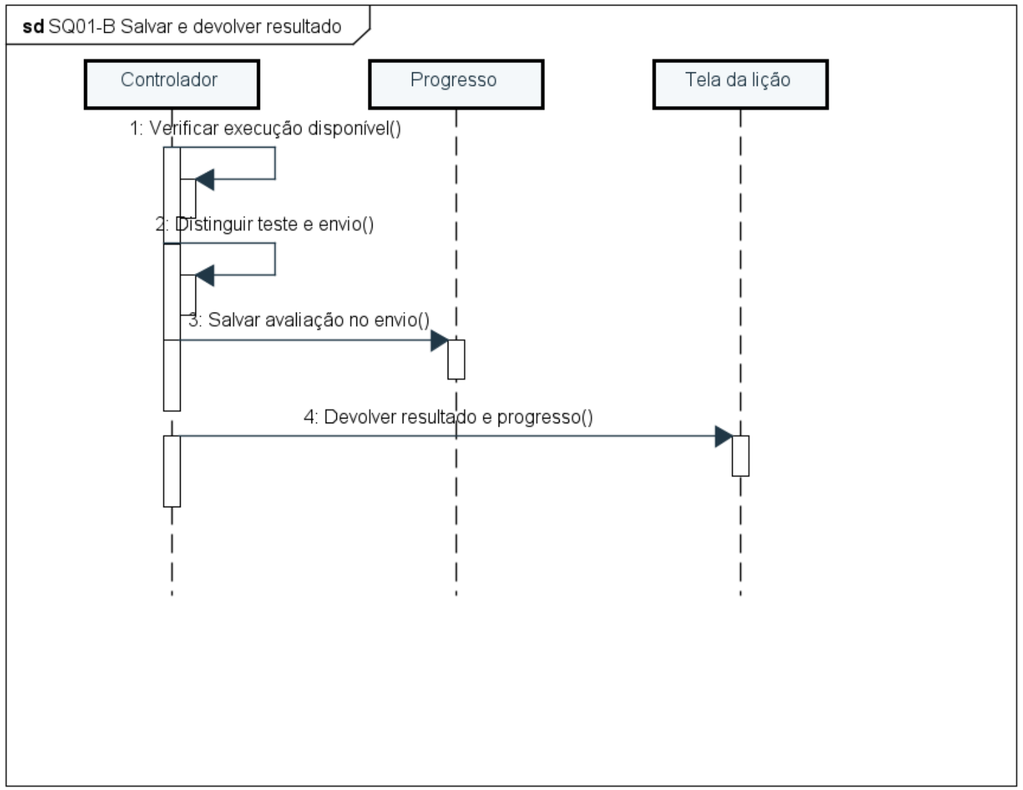
```
17  const parsed = lessonAnswerSchema.safeParse(await request.json().catch(()
    => null));
18  if (!parsed.success)
19    throw new ValidationError('INVALID_ANSWER', 'Confira os dados da
    resposta e tente novamente.');
20  const answer = parsed.data;
21  const lesson = getLessonById(lessonId);
22  const step = lesson?.steps.find((s) => s.id === answer.stepId);
23  if (!lesson || !step) throw new AppError('LESSON_NOT_FOUND', 'Atividade
    não encontrada.', 404);
24  await requireLessonAccess(user.id, lesson);
25  const outcome = await assessLessonStep(lesson, step, answer);
```

Validar resposta, localizar lição e etapa, verificar acesso e solicitar avaliação.

Persistir conclusão e devolver o resultado

SQ01-B Recorte da sequência e trecho correspondente

Interação



Implementação

src/app/api/lessons/[lessonId]/attempt/route.ts

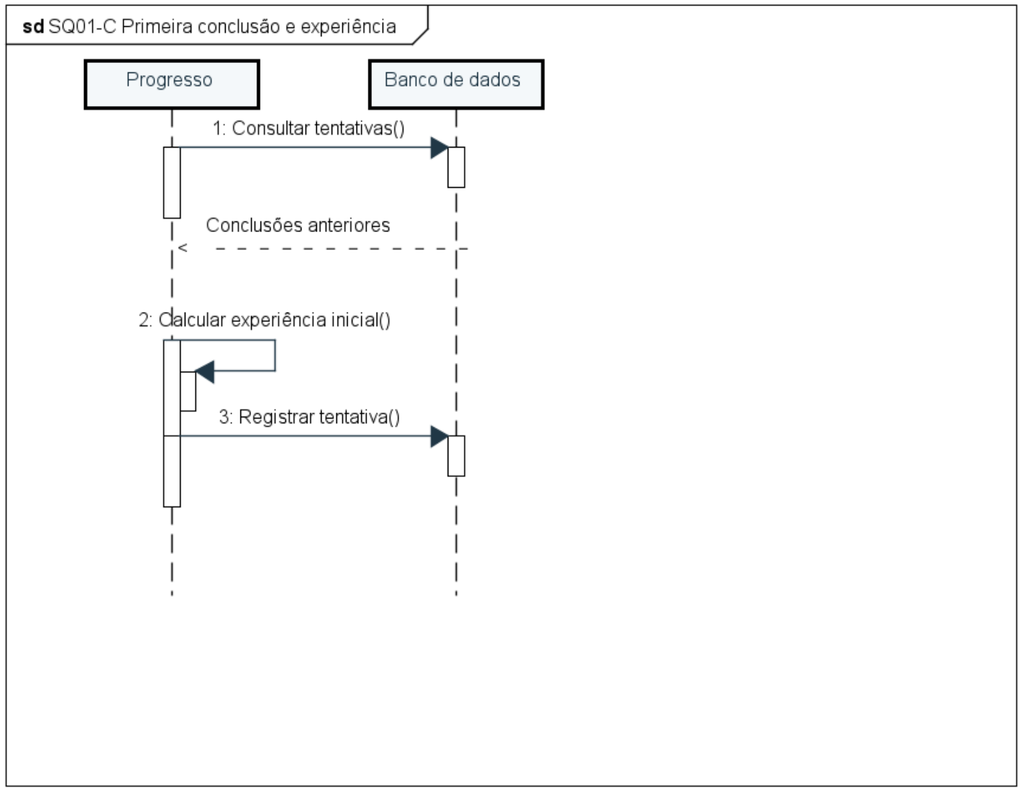
```
26 if ('unavailable' in outcome && outcome.unavailable)
27   throw new AppError(
28     'RUNNER_UNAVAILABLE',
29     'Serviço de execução indisponível. Seu progresso não foi alterado;
   tente novamente.',
30     503
31   );
32 if (answer.action === 'run') return NextResponse.json({ ...outcome,
   xpEarned: 0 });
33 const progress = await saveLessonAssessment(
34   user.id,
35   lesson,
36   step,
37   outcome.isCorrect,
38   answer.selectedOption ?? 0
39 );
40 return NextResponse.json({ ...outcome, ...progress });
```

Tratar indisponibilidade, separar teste de envio e persistir avaliação antes da resposta.

Conceder XP somente na primeira conclusão

SQ01-C Recorte da sequência e trecho correspondente

Interação



Implementação

src/lib/learning/lesson-progress.ts

```
55 const where = { user_id_quiz_id: { user_id: userId, quiz_id: step.id } };
56 const attempts = await tx.quizAttempt.findMany({
57   where: { user_id: userId, quiz_id: { in: lesson.steps.map((s) => s.id) }
   },
58   select: { quiz_id: true, is_correct: true },
59 });
60 const previous = attempts.find((attempt) => attempt.quiz_id === step.id);
61 const firstCompletion = isCorrect && !previous?.is_correct;
62 const xpEarned = firstCompletion ? Math.max(0, step.xp) : 0;
   ⋮ linhas intermediárias omitidas
75 if (!previous)
76   await tx.quizAttempt.create({
77     data: {
78       user_id: userId,
79       quiz_id: step.id,
80       selected_index: selectedIndex,
81       is_correct: isCorrect,
82       xp_earned: xpEarned,
83     },
84   });
```

Localizar conclusão anterior e gravar a tentativa com XP calculado na transação.

Demonstração prática

Pedro Antonio Silvestre Tescaro Aprendizagem e conclusão de atividades da lição

Preparação

Usar conta de demonstração com progresso conhecido e anotar o XP inicial. Deixar aberta a lição de catálogo /lesson/js-l1, sem o parâmetro mode=workspace.

Execução

Mostrar o enunciado e a resposta preparada da atividade da lição. Acionar Testar e explicar que a resposta é avaliada no servidor sem conceder XP.

Resultado esperado

Ação run retorna xpEarned igual a zero. A conclusão correta inédita pode conceder o XP da etapa.

Conferência

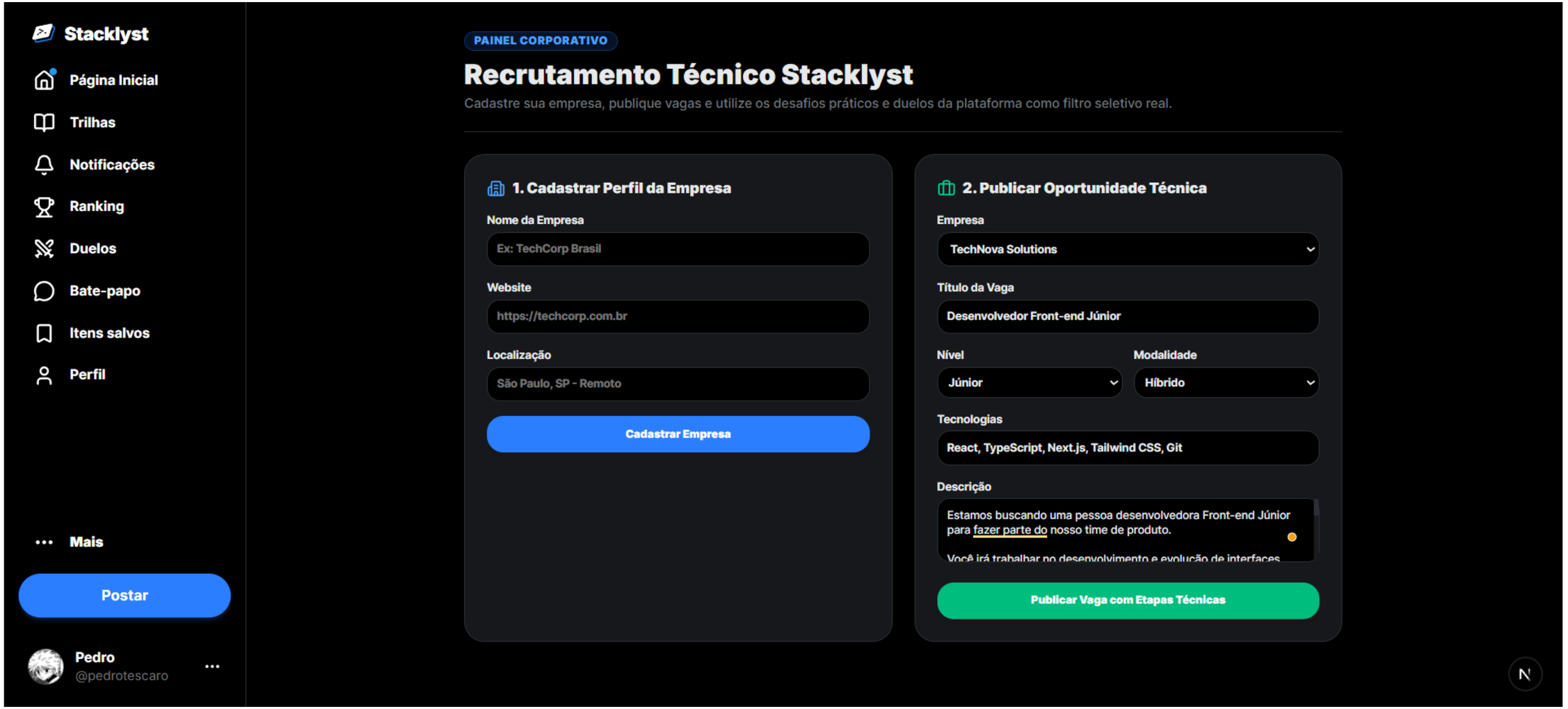
Acionar a verificação definitiva da solução e observar o resultado e o progresso. Comparar o XP ao valor anterior e repetir a verificação para mostrar que a mesma conclusão não concede XP novamente.

Continuidade

O perfil do candidato já existe. A próxima demonstração cria a oportunidade que será consumida no último bloco.

Guilherme: publicação de vaga

Guilherme Alves de Campos UC017



No protótipo: empresa autorizada, dados da oportunidade e comando de publicação.

Requisitos e regras na interface

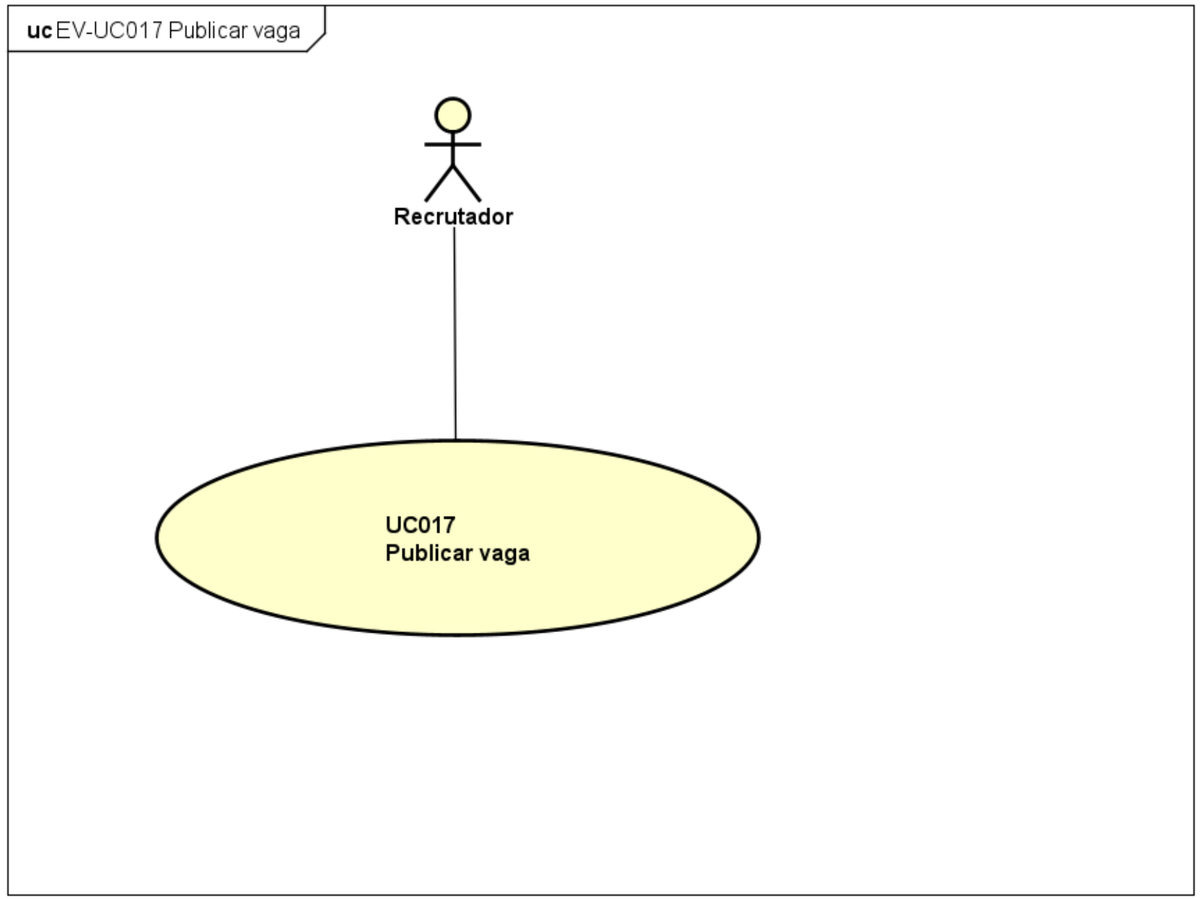
UC017 Publicar uma vaga

Requisitos funcionais	Requisitos não funcionais	Regras de negócio
RF035 Publicar vaga Criar vaga com requisitos e etapas para uma empresa autorizada. RF036 Consultar vagas Listar e filtrar oportunidades abertas.	RNF001 Autenticação protegida Verificar a sessão no servidor antes de acessar rotas privadas. RNF002 Autorização por papel Validar o papel, o acesso à empresa e os vínculos dos recursos.	RN020 Publicação de vaga Permitir publicação somente a recrutador ou administrador com acesso à empresa. RN017 Controle por papel Aplicar as permissões correspondentes ao papel do usuário.

RF: o que a tela permite fazer. RNF: qualidade e restrições. RN: condições do domínio.

Casos de uso da funcionalidade

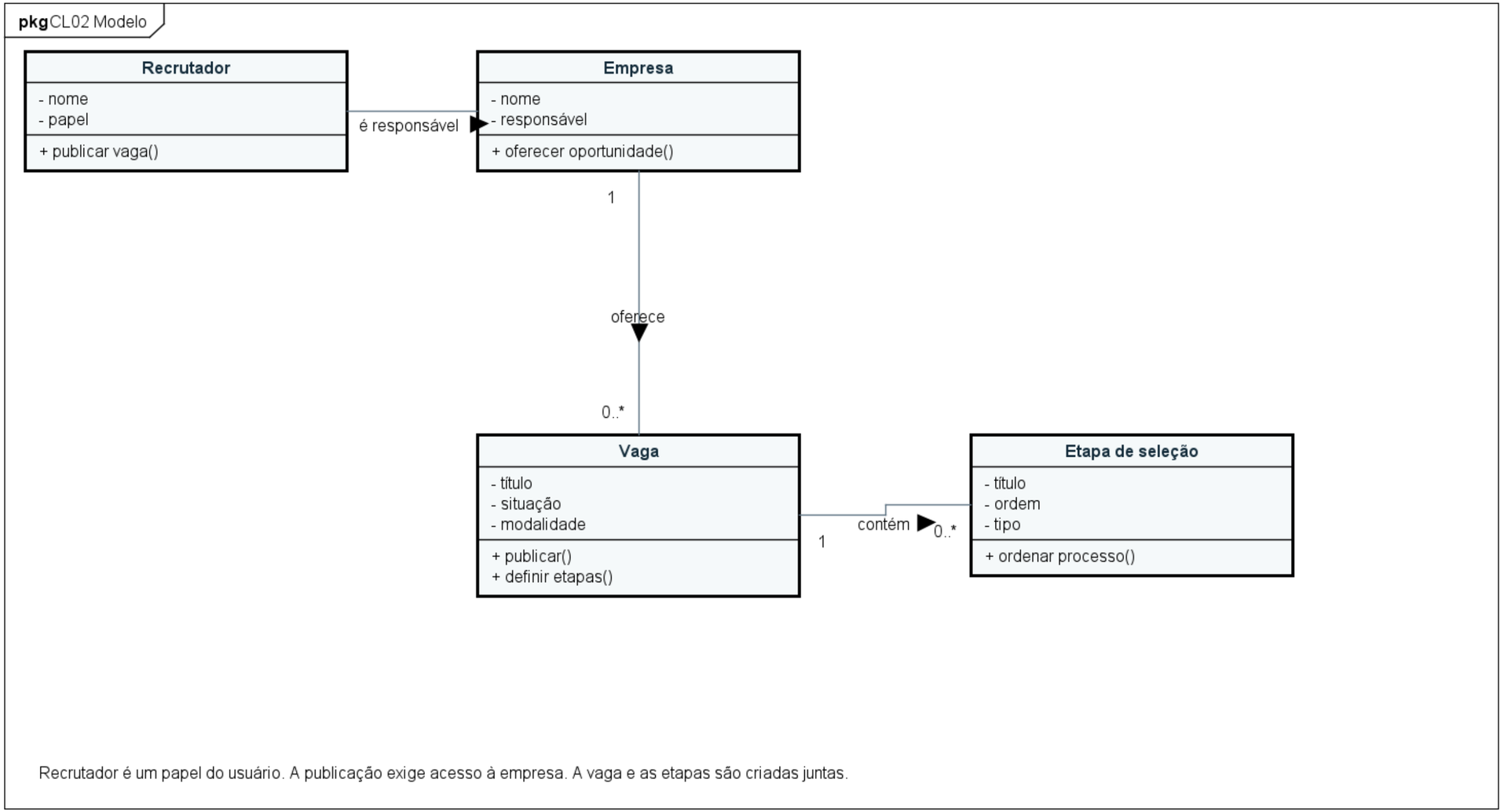
UC017 Publicar uma vaga



Sessão autenticada como pré-condição dos fluxos apresentados.

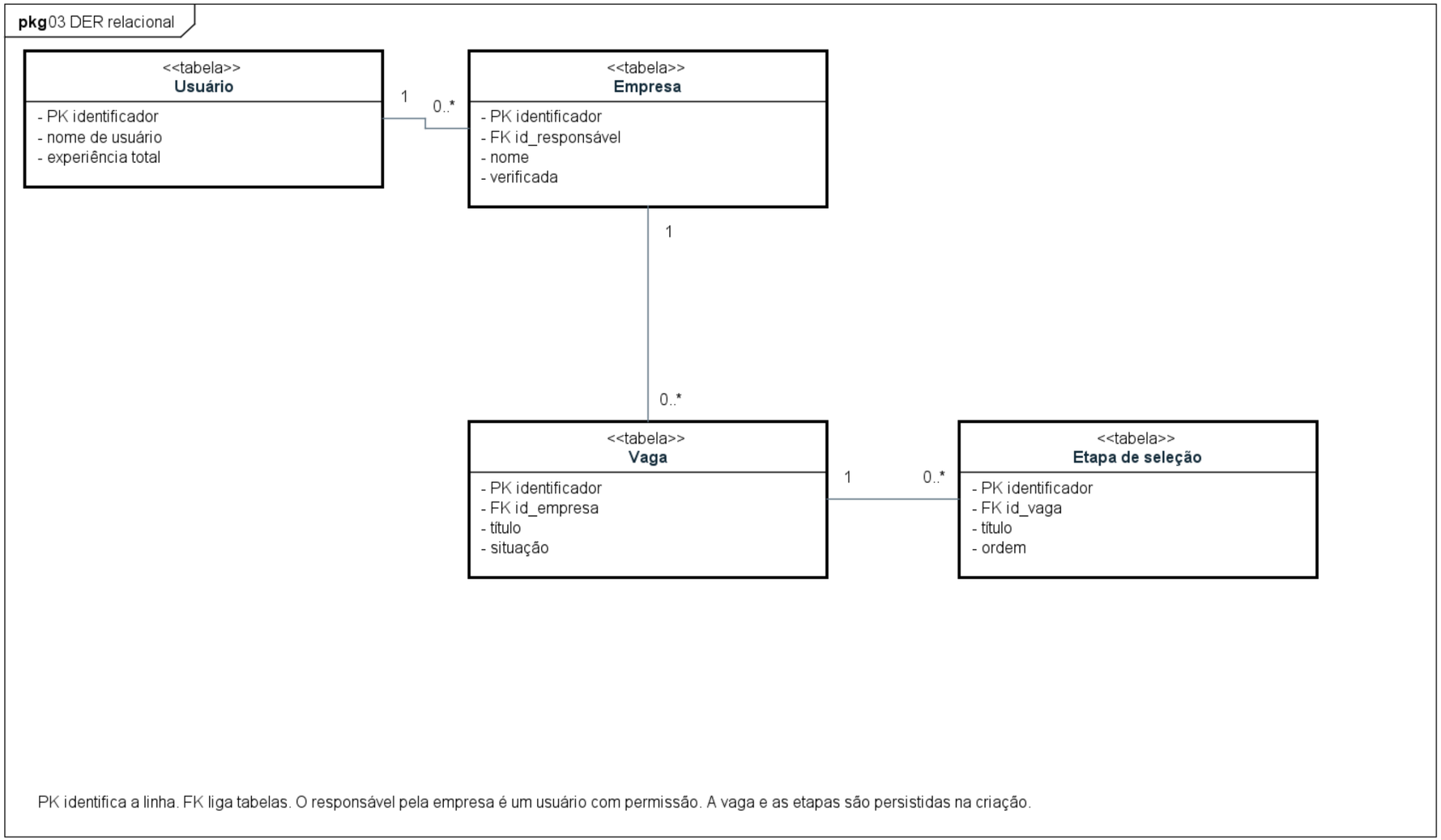
Classes da publicação de vaga

UC017 Empresa, vaga e etapas



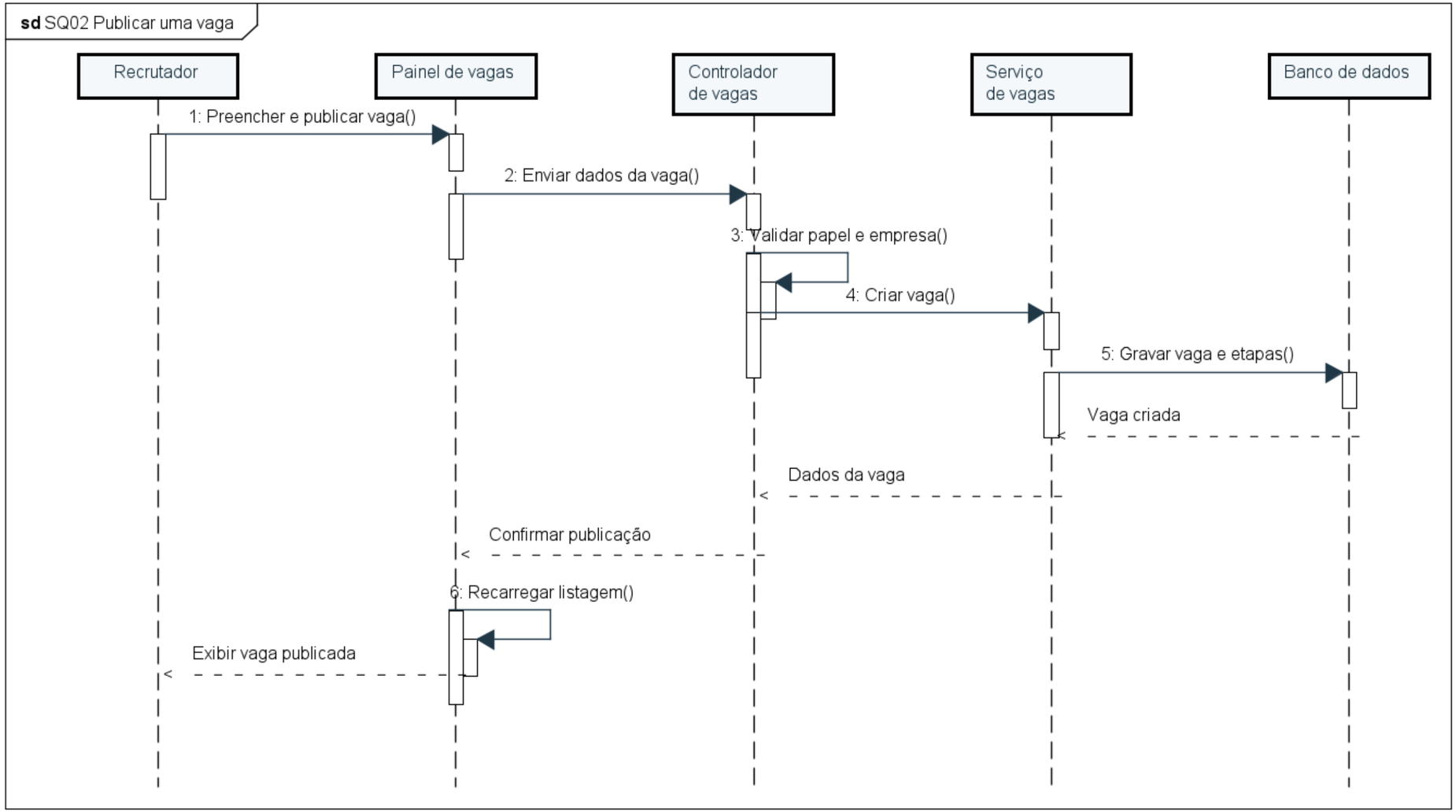
Banco de dados da publicação

UC017 Chaves e vínculos



Sequência completa: publicar vaga

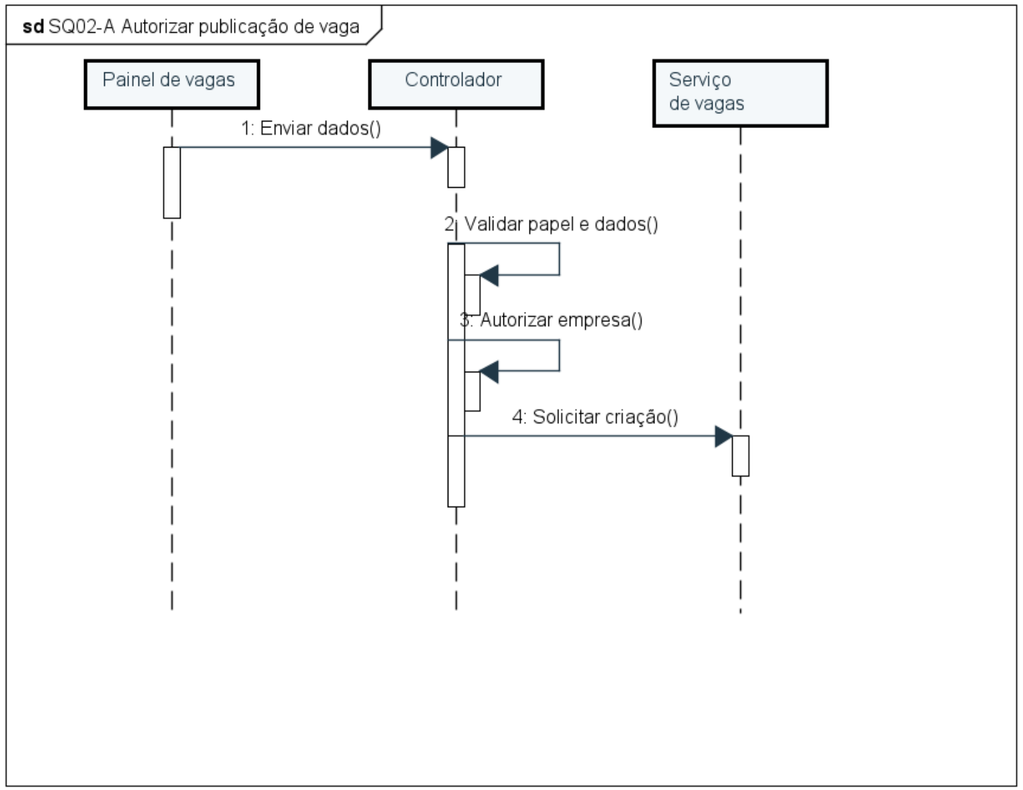
UC017 Da autorização à confirmação



Autorizar a publicação da vaga

SQ02-A Recorte da sequência e trecho correspondente

Interação



Implementação

src/app/api/jobs/route.ts

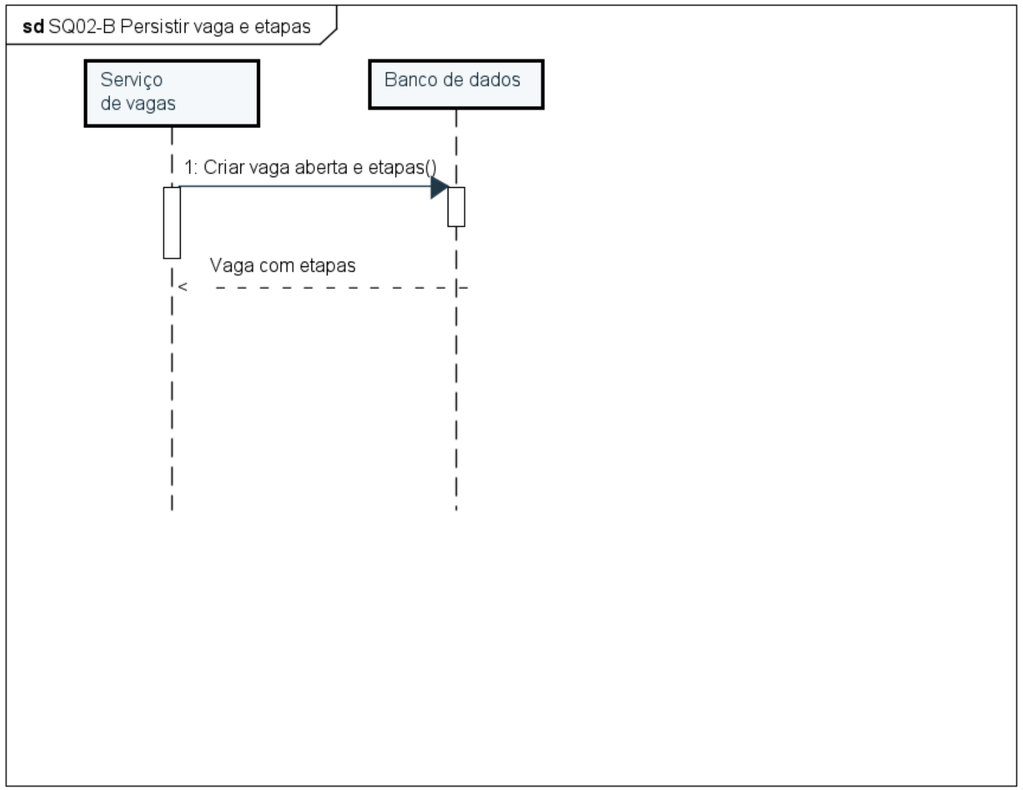
```
67  export const POST = apiHandler(async (req) => {
68    const user = await requireRole(['RECRUITER', 'ADMIN']);
69    const body = await req.json();
70    const parsed = createJobSchema.parse(body);
71    await requireCompanyAccess(parsed.company_id, user);
72
73    const job = await JobService.createJob({
74      companyId: parsed.company_id,
75      title: parsed.title,
76      description: parsed.description,
77      level: parsed.level,
78      technologies: parsed.technologies,
```

Exigir recrutador ou administrador, validar dados e vínculo com a empresa.

Gravar a vaga e suas etapas

SQ02-B Recorte da sequência e trecho correspondente

Interação



Implementação

src/services/job.service.ts

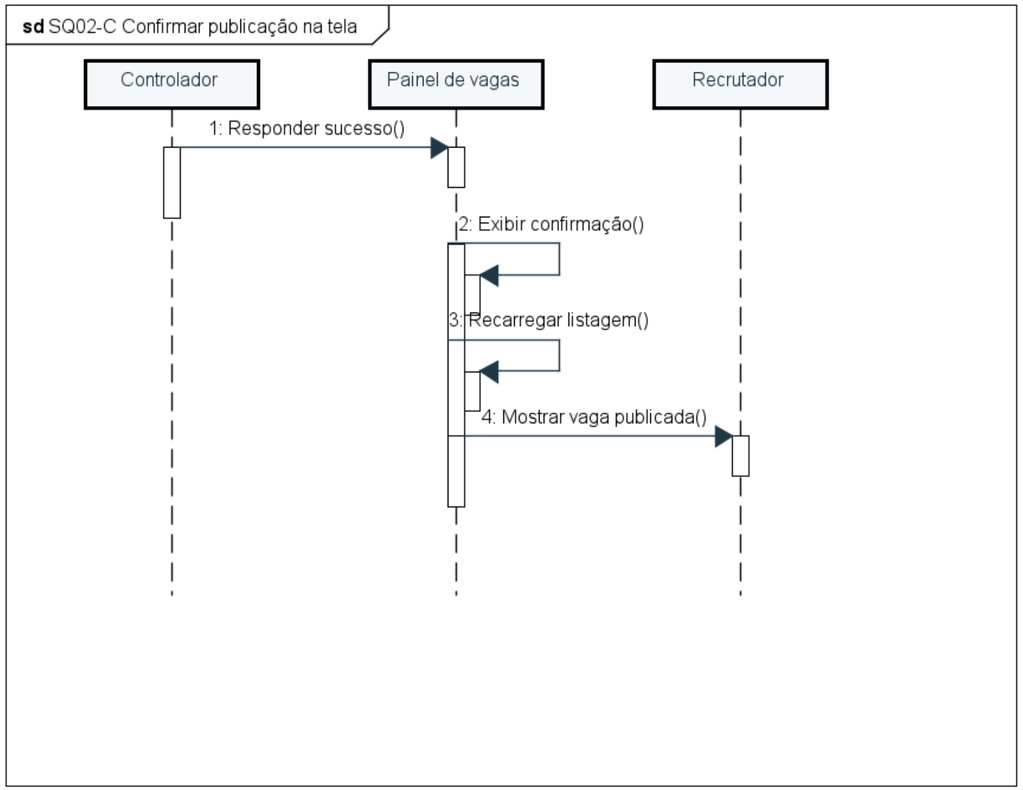
```
215 const job = await prisma.job.create({
216   data: {
217     company_id: companyId,
218     title,
219     description,
220     level,
221     technologies,
222     modality,
    : linhas intermediárias omitidas
229 status: 'OPEN',
230 stages: {
231   create: defaultStages.map((st) => ({
232     title: st.title,
233     type: st.type,
234     order: st.order,
235     challenge_id: st.challengeId,
236     description: st.description,
237   })),
238 },
```

Gravar vaga aberta e etapas do processo na mesma operação relacional.

Confirmar a publicação na interface

SQ02-C Recorte da sequência e trecho correspondente

Interação



Implementação

src/app/recruiter/RecruiterContent.tsx

```
294    if (res.ok) {
295      setSuccessMsg('Vaga publicada com sucesso e vinculada às etapas de
     testes do Stacklyst!');
296      setJobTitle('');
297      setJobDescription('');
298      loadJobs(); // Instantly update jobs list
299      setTimeout(() => setSuccessMsg(null), 4000);
300    } else {
301      const errData = await res.json().catch(() => null);
302      setErrorMsg(errData?.error || 'Erro ao publicar vaga.');
303    }
304  } catch (err) {
305    console.error(err);
306    setErrorMsg('Erro de conexão ao publicar vaga.');
```

Depois do retorno da API, confirmar publicação e recarregar a listagem.

Demonstração prática

Guilherme Alves de Campos Publicação de vaga

Preparação

Deixar a conta de recrutador autenticada e vinculada à empresa de demonstração. Usar /recruiter com empresa cadastrada previamente.

Execução

Identificar no protótipo a empresa e os dados obrigatórios da vaga. Publicar a vaga e mostrar a confirmação na interface.

Resultado esperado

O servidor exige papel de recrutador ou administrador e acesso à empresa. A vaga é persistida como aberta e recebe suas etapas de recrutamento.

Conferência

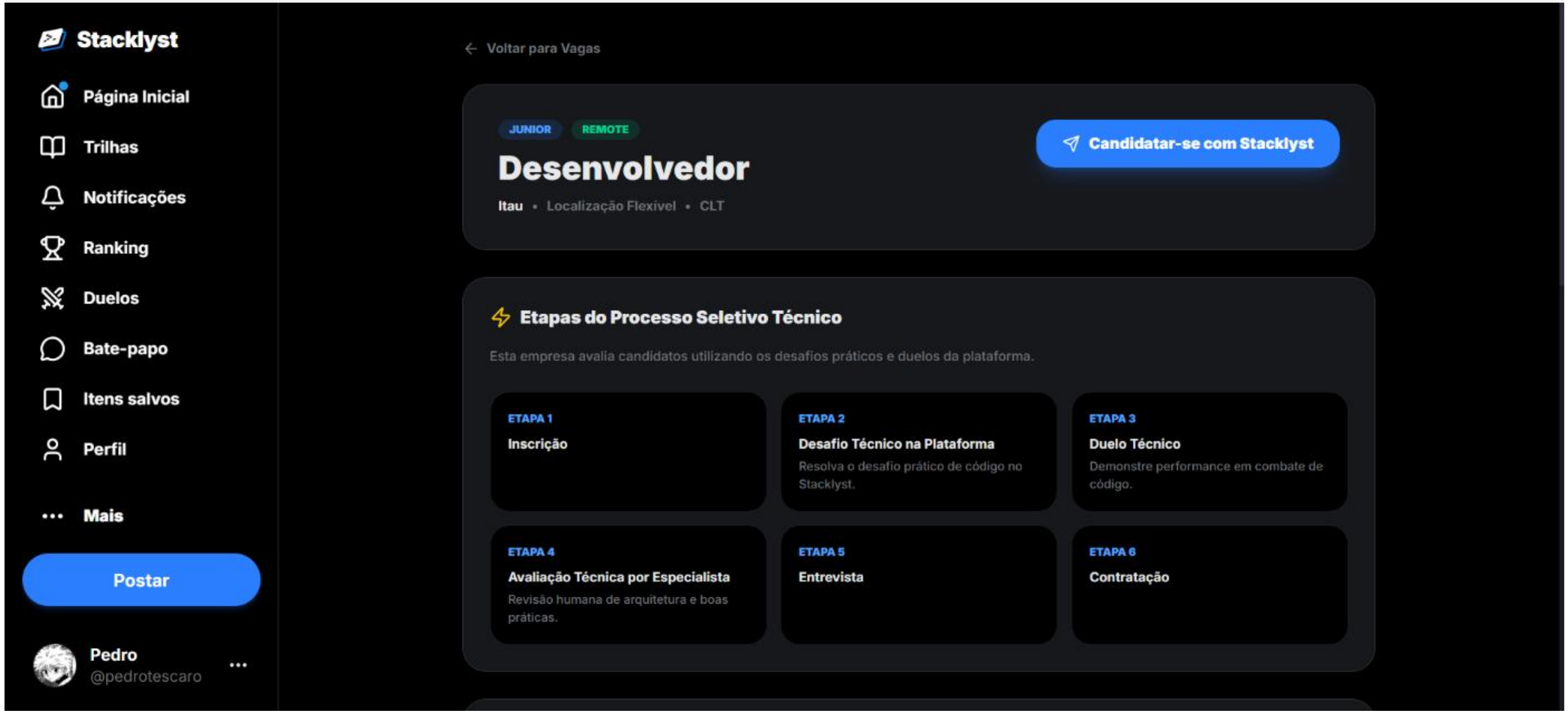
Localizar a vaga na listagem atualizada e observar as etapas associadas. Abrir /jobs e a página da vaga para preparar a continuidade da demonstração.

Continuidade

A vaga criada e sua primeira etapa já estão disponíveis para a candidatura do terceiro bloco.

Paulo: candidatura à vaga

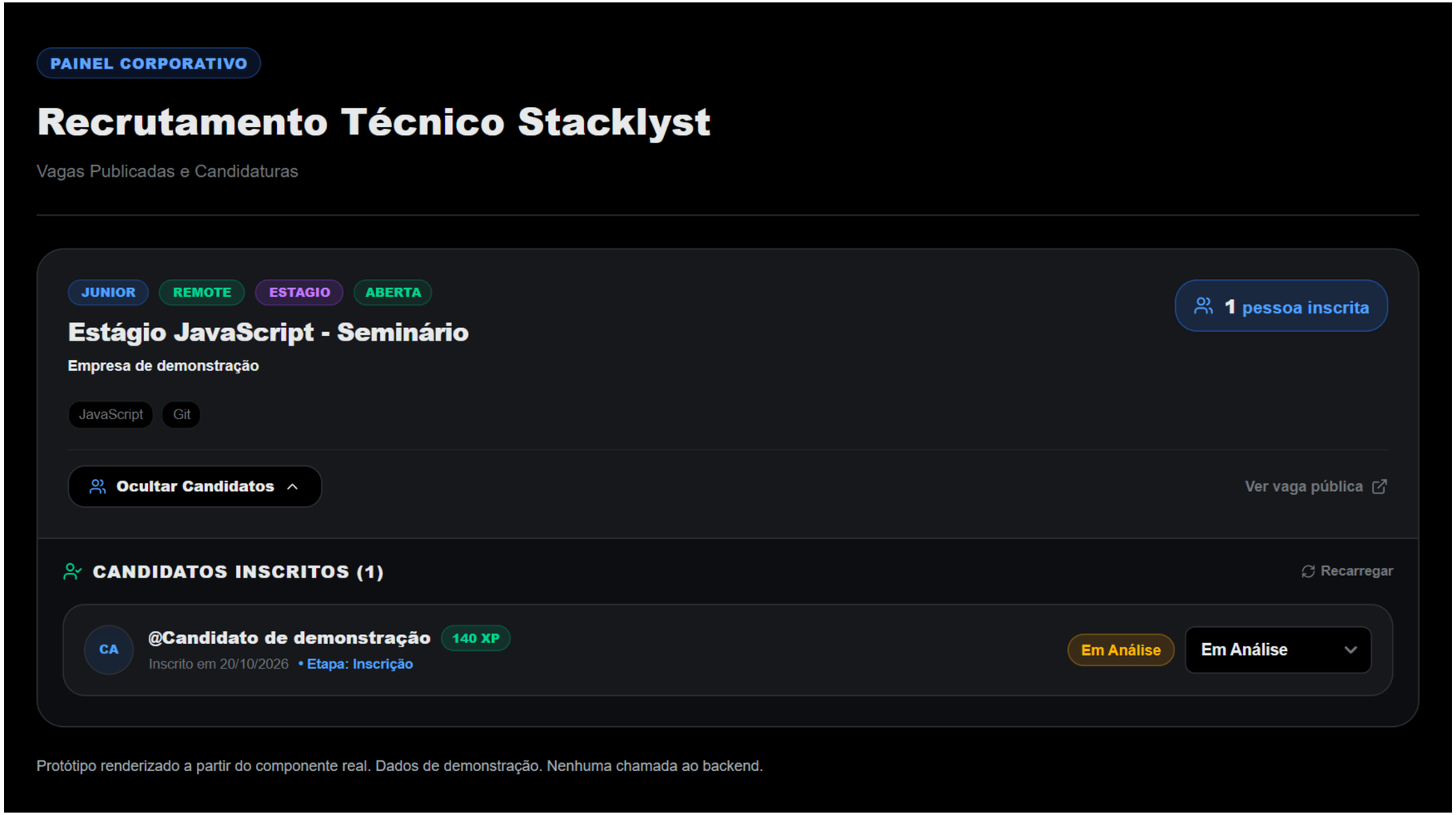
Paulo Henrique Leal dos Santos UC022



No protótipo: oportunidade aberta e comando de candidatura com o perfil do usuário.

Protótipo do painel de candidaturas

Paulo Henrique Leal dos Santos UC026



Dados de demonstração: candidato, situação da candidatura e seletor do recrutador.

Requisitos e regras na interface

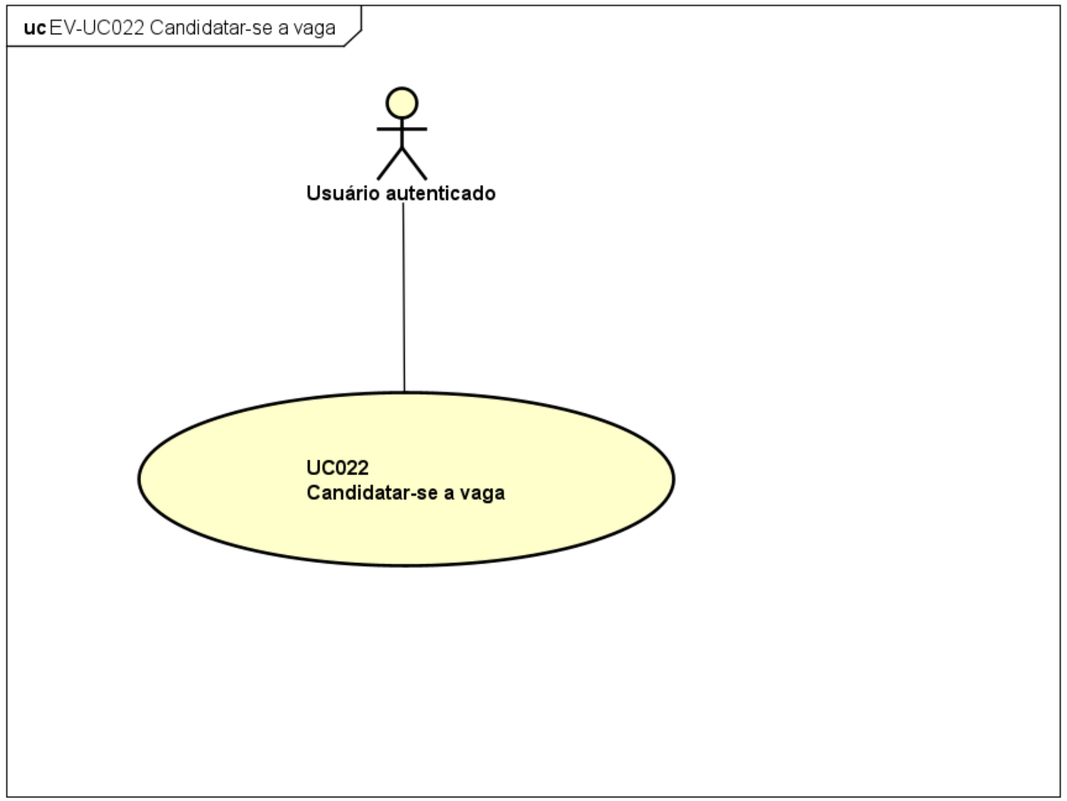
UC022 e UC026 Candidatar-se e acompanhar a seleção

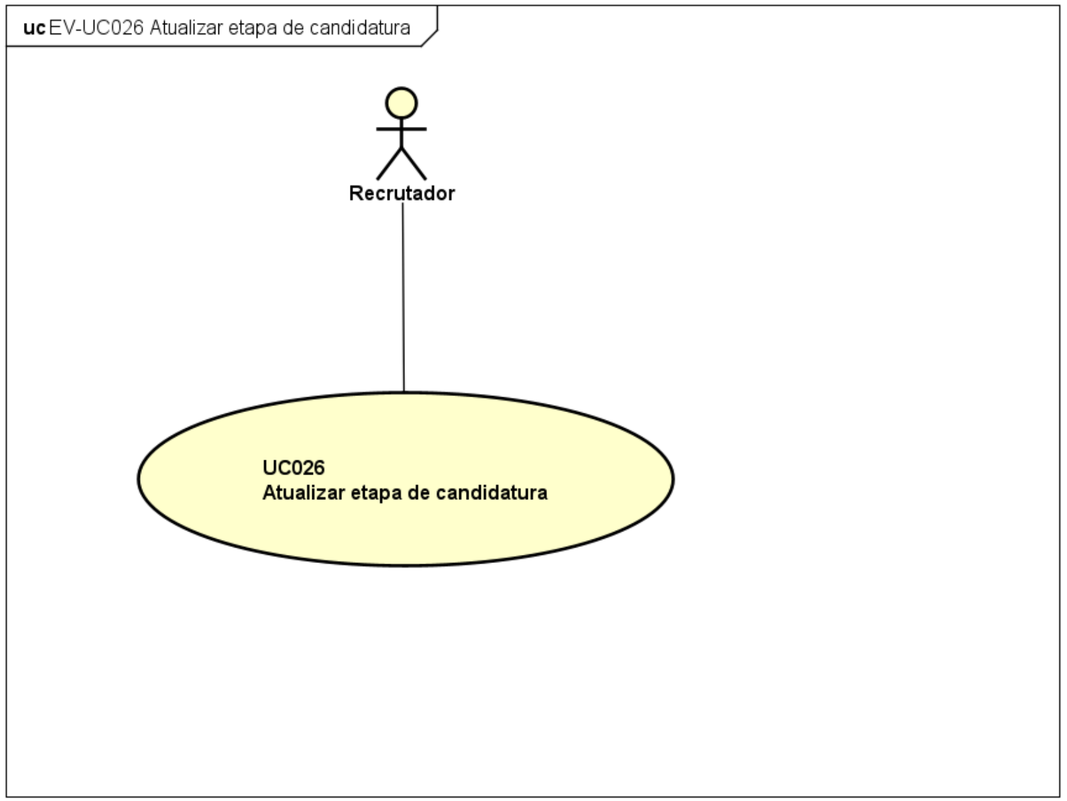
Requisitos funcionais	Requisitos não funcionais	Regras de negócio
RF047 Candidatar-se a vaga Registrar uma candidatura do usuário a uma vaga aberta, na etapa inicial. RF048 Atualizar etapa de candidatura Atualizar etapa, status e feedback e notificar o candidato associado. RF044 Gerar notificação Gerar a notificação ao destinatário após a operação correspondente.	RNF002 Autorização por papel Validar o papel, o acesso à empresa e os vínculos dos recursos. RNF005 Separação público/privado Retornar somente campos autorizados no perfil e no recrutamento.	RN026 Candidatura a vaga Aceitar uma candidatura por usuário e vaga aberta. A atualização exige responsável autorizado. RN022 Notificações Notificar somente o candidato associado. Falha da notificação não desfaz a atualização.

RF: o que a tela permite fazer. RNF: qualidade e restrições. RN: condições do domínio.

Casos de uso da funcionalidade

UC022 e UC026 Candidatar-se e acompanhar a seleção

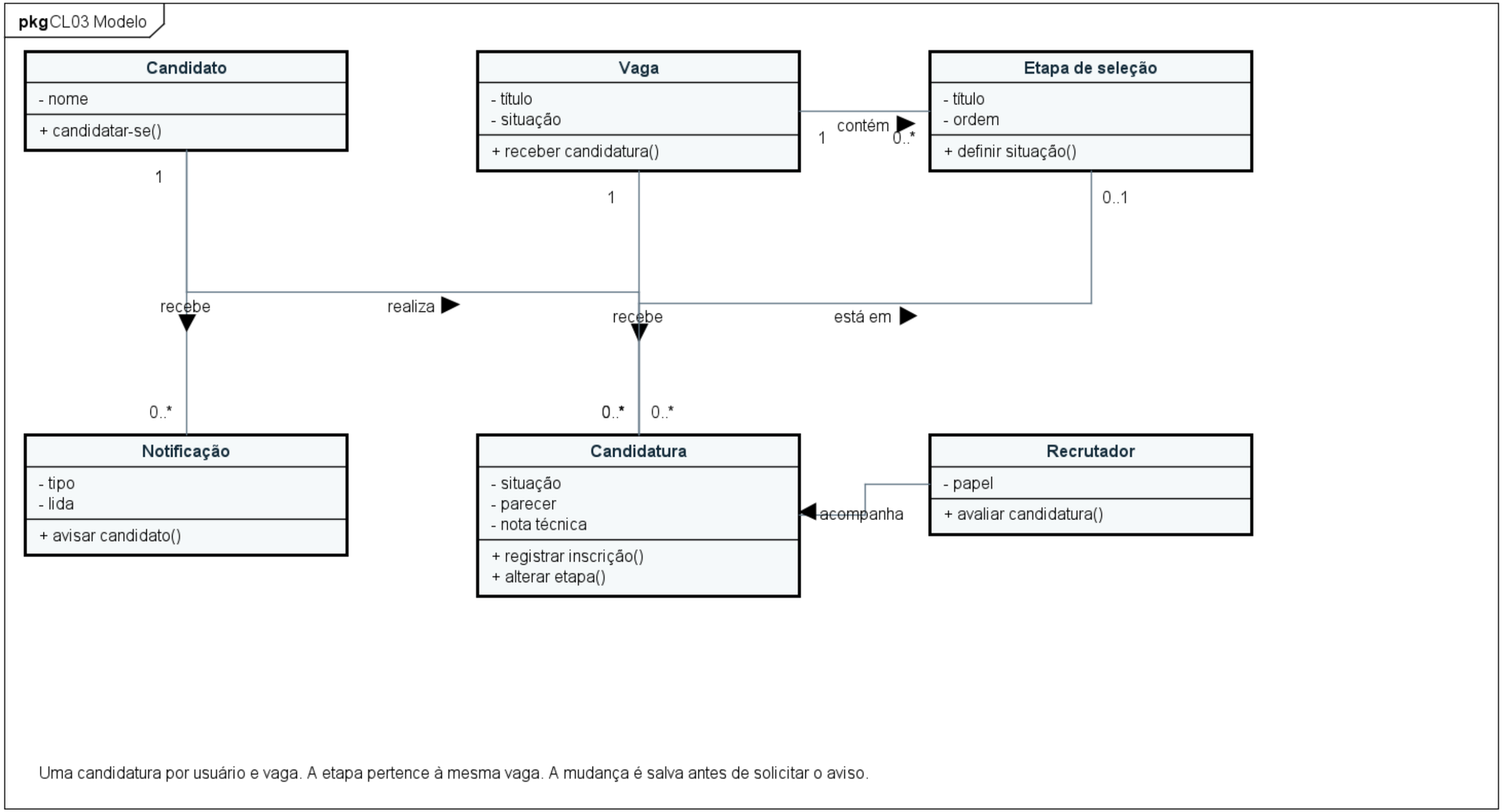




Sessão autenticada como pré-condição dos fluxos apresentados.

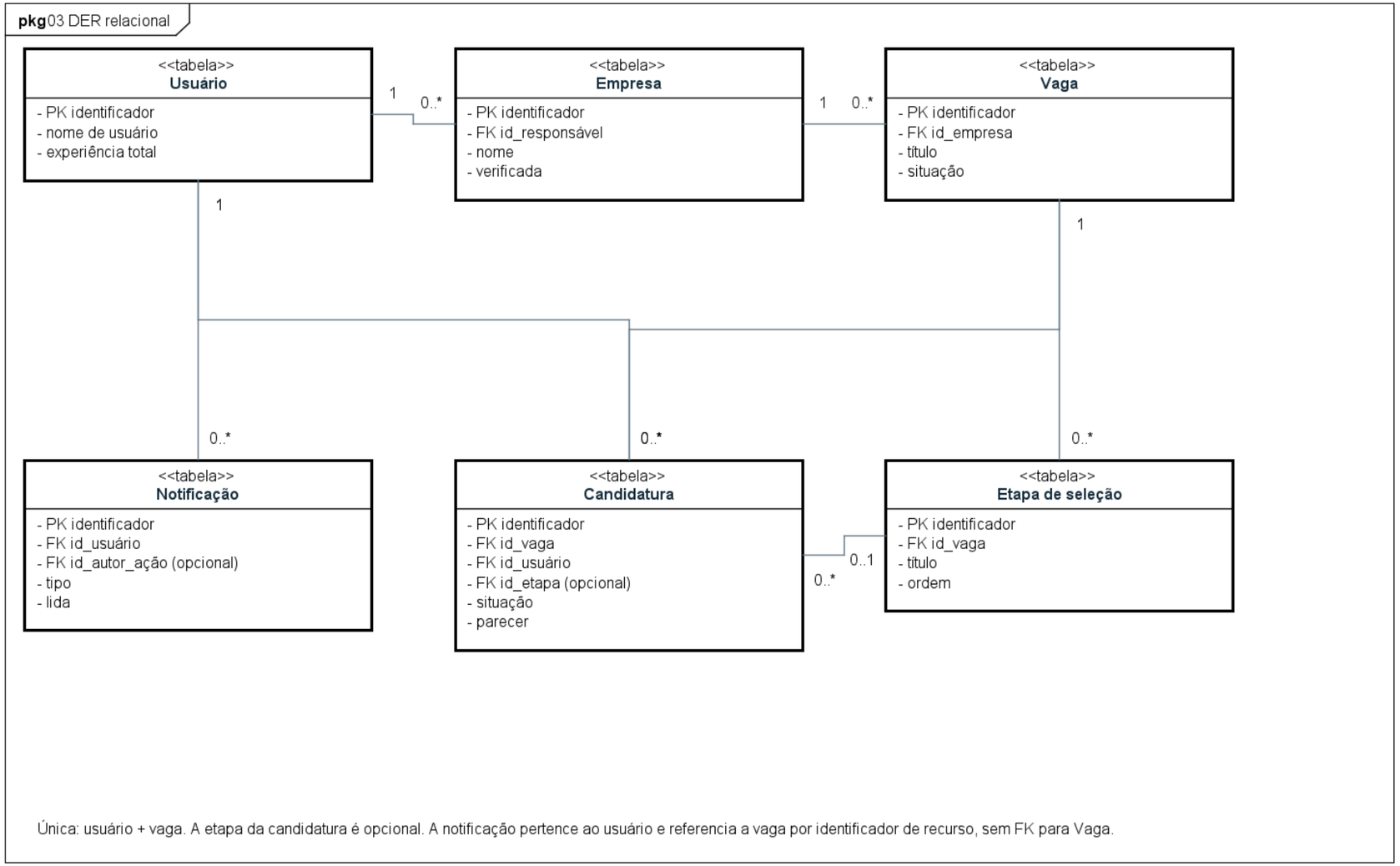
Classes da candidatura e do retorno

UC022 e UC026 Vínculos e responsabilidades



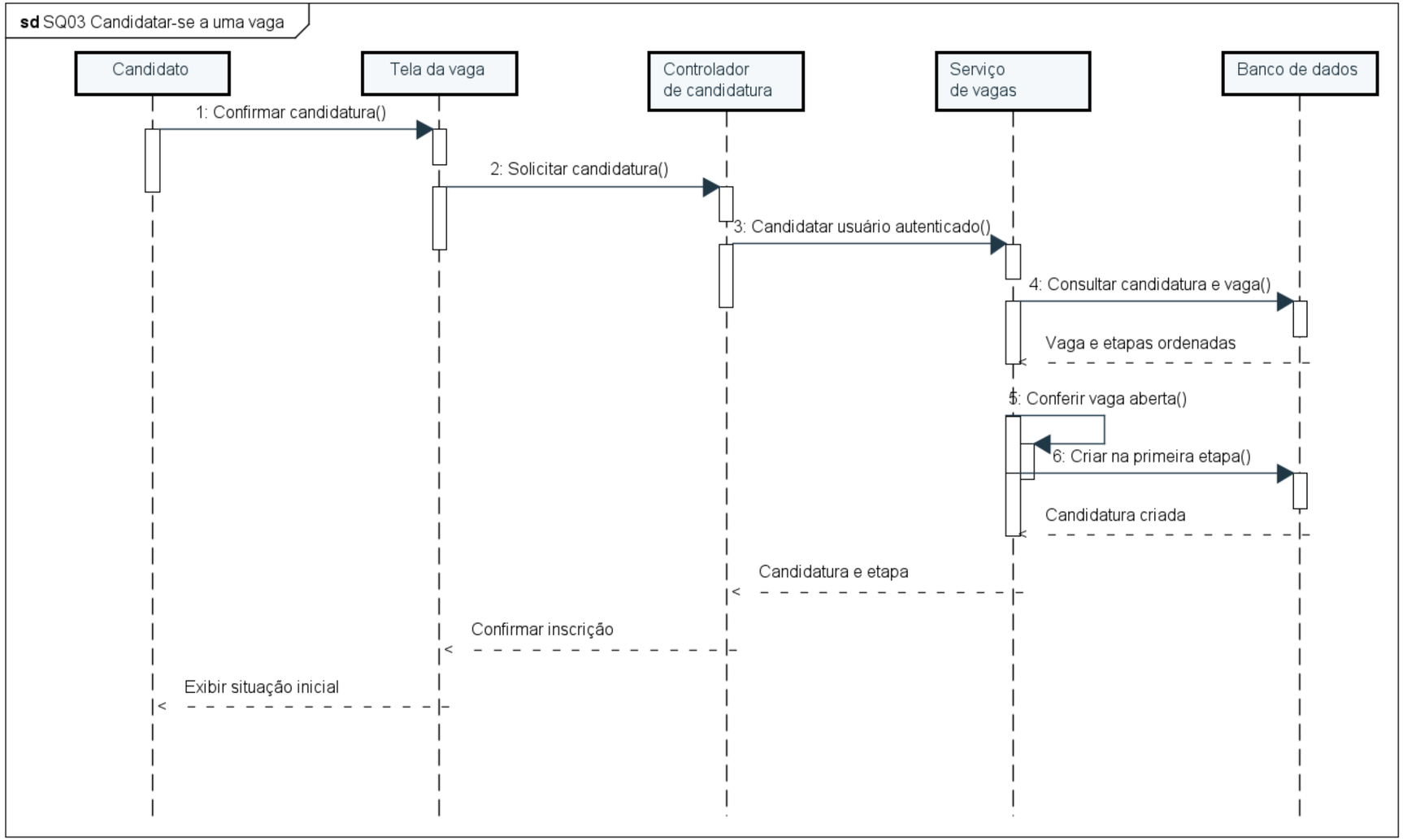
Banco de dados da candidatura

UC022 e UC026 Integridade dos vínculos



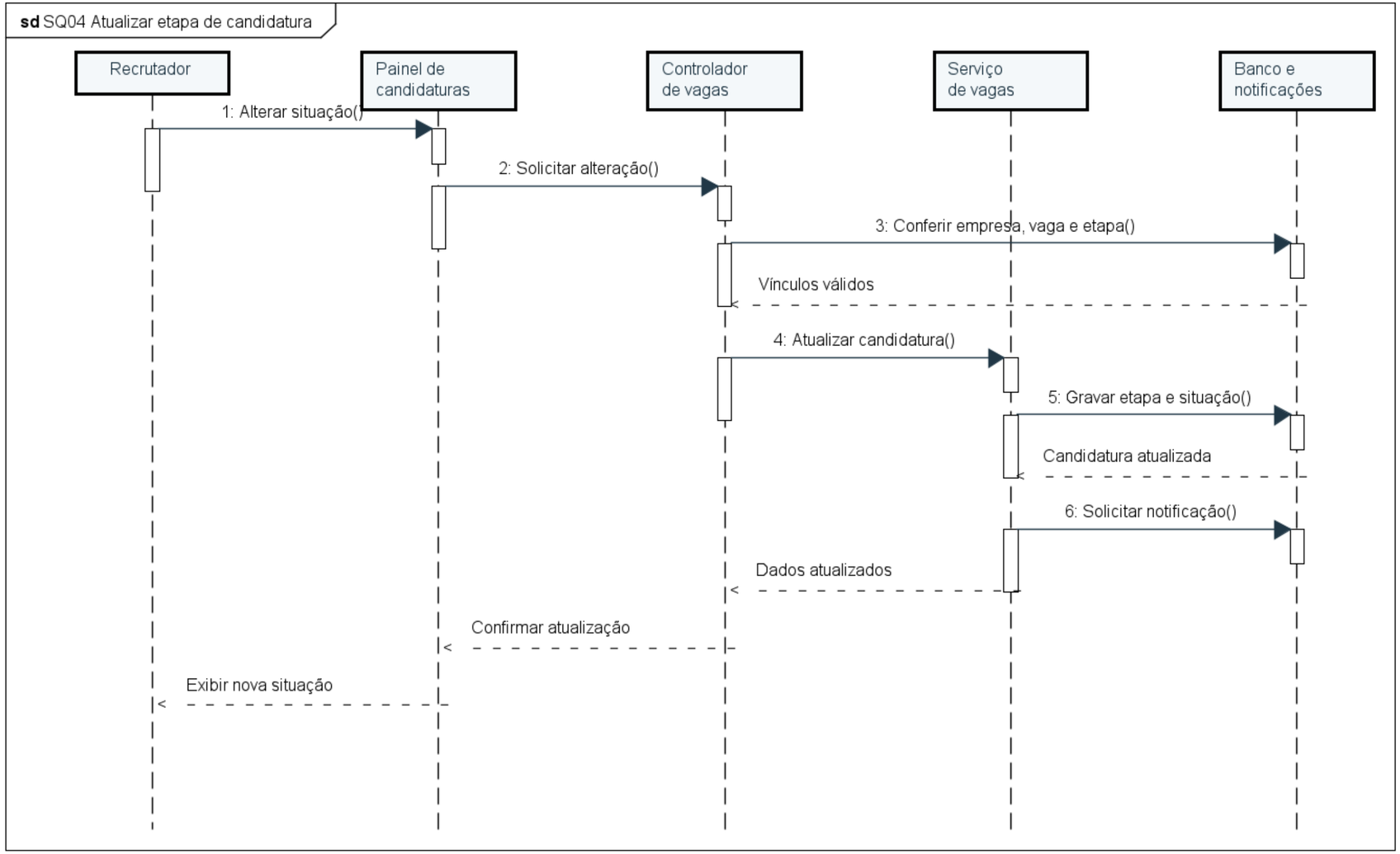
Sequência completa: candidatar-se

UC022 Da vaga aberta à etapa inicial



Sequência completa: atualizar candidatura

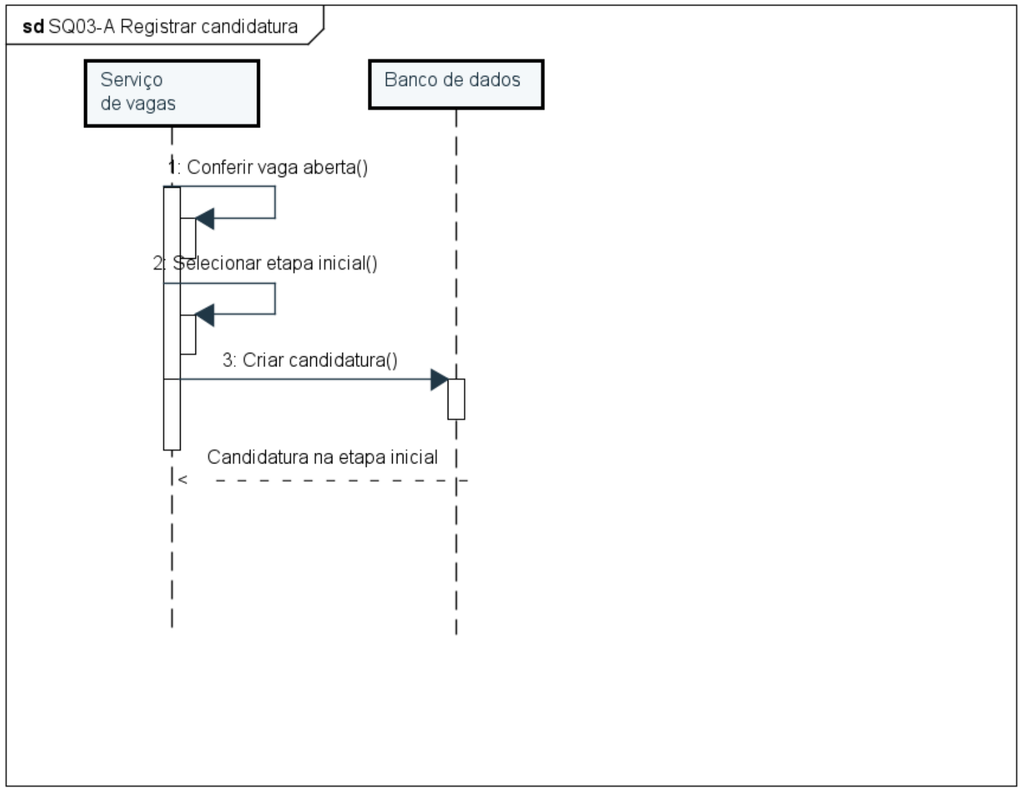
UC026 Situação e notificação ao candidato



Criar candidatura na primeira etapa

SQ03-A Recorte da sequência e trecho correspondente

Interação



Implementação

src/services/job.service.ts

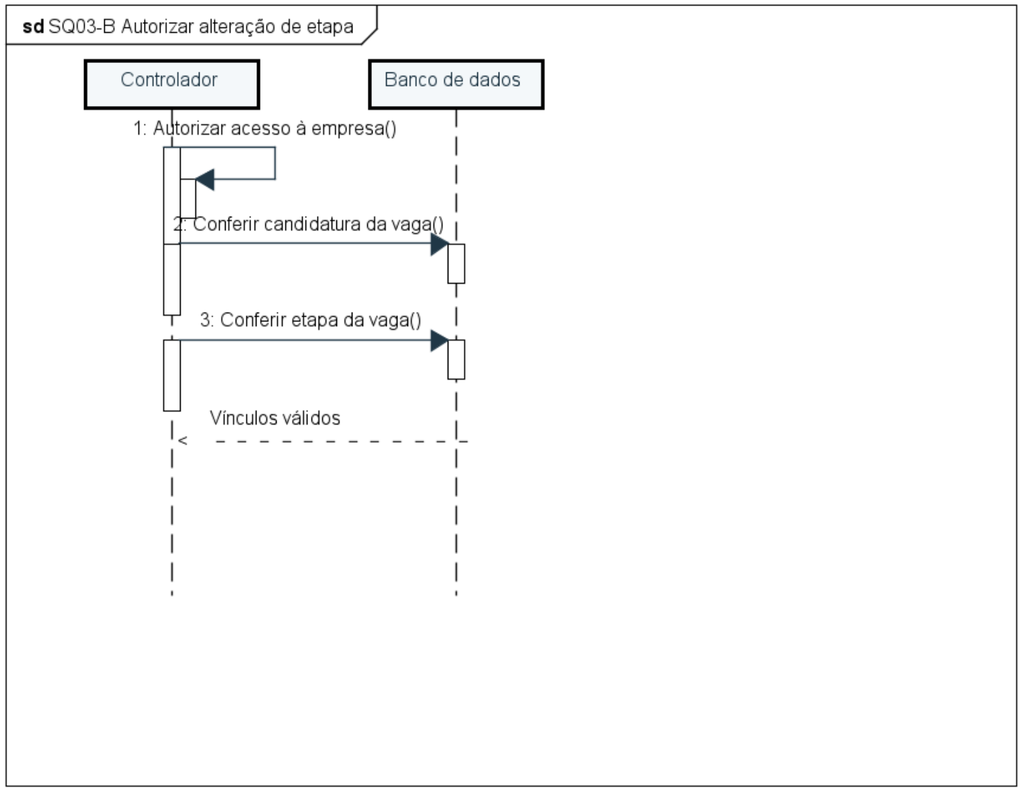
```
275 if (!job || job.status !== 'OPEN') {
276   throw new Error('Esta vaga não está mais aceitando inscrições.');
277 }
278
279 const firstStage = job.stages[0];
280
281 const application = await prisma.jobApplication.create({
282   data: {
283     job_id: jobId,
284     user_id: userId,
285     stage_id: firstStage?.id,
286     status: 'APPLIED',
287   },
288   include: {
289     stage: true,
290     job: { include: { company: true } },
291   },
292 });
```

Aceitar apenas vaga aberta e relacionar candidatura ao usuário e à primeira etapa.

Conferir a candidatura e a etapa escolhida

SQ03-B Recorte da sequência e trecho correspondente

Interação



Implementação

src/app/api/jobs/[id]/route.ts

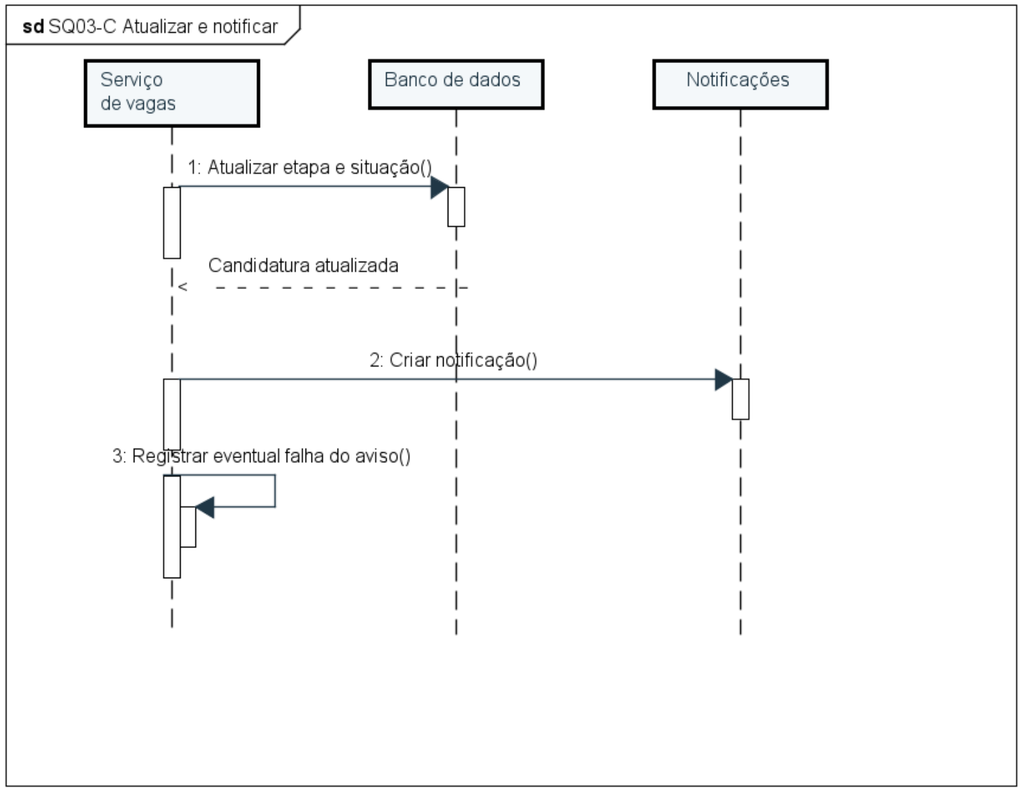
```
30 export const PATCH = apiHandler(async (req, { params }) => {
31   const user = await requireRole(['RECRUITER', 'ADMIN']);
32   const { id } = await params;
33   const job = await prisma.job.findUniqueOrThrow({ where: { id } });
34   await requireCompanyAccess(job.company_id, user);
35   const body = await req.json();
36   const parsed = updateStageSchema.parse(body);
37   const application = await prisma.jobApplication.findFirst({
38     where: { id: parsed.application_id, job_id: id },
39   });
40   if (!application)
41     return NextResponse.json({ error: 'Candidatura não encontrada nesta
   vaga.' }, { status: 404 });
42   if (
43     parsed.stage_id &&
44     !(await prisma.jobRecruitmentStage.findFirst({ where: { id:
   parsed.stage_id, job_id: id } }))
45   )
46     return NextResponse.json({ error: 'Etapa inválida para esta vaga.' },
   { status: 400 });
```

Autorizar a empresa e impedir candidatura ou etapa pertencente a outra vaga.

Atualizar a candidatura e avisar o candidato

SQ03-C Recorte da sequência e trecho correspondente

Interação



Implementação

src/services/job.service.ts

```
310 const application = await prisma.jobApplication.update({
311   where: { id: applicationId },
312   data: {
313     stage_id: stageId,
314     status,
315     feedback,
316     technical_score: technicalScore,
317   },
    : linhas intermediárias omitidas
324 try {
325   await NotificationService.create({
326     userId: application.user_id,
327     type: 'JOB_APPLICATION_UPDATE',
328     resourceId: application.job_id,
329     resourceType: 'JOB',
330   });
331 } catch (err) {
332   logger.error('Failed to notify job application update', { error:
    String(err) });
333 }
```

Persistir etapa e situação e depois solicitar notificação ao candidato. Falha do aviso é registrada e não desfaz a atualização.

Demonstração prática

Paulo Henrique Leal dos Santos Candidatura, acompanhamento e retorno ao candidato

Preparação

Usar a vaga criada no bloco anterior e uma conta de candidato que ainda não se candidatou a ela. Manter duas sessões separadas, candidato e recrutador, para evitar pausas de autenticação.

Execução

Na sessão do candidato, abrir a vaga e acionar Candidatar-se com Stacklyst. Mostrar a confirmação da candidatura na etapa inicial.

Resultado esperado

A candidatura referencia a vaga e o candidato e começa na primeira etapa disponível. Uma candidatura repetida é rejeitada e vagas fechadas não aceitam inscrição.

Conferência

Na sessão do recrutador, carregar as candidaturas da vaga e localizar o mesmo candidato. No seletor da candidatura, alterar a situação de Inscrito para Em Análise e mostrar a confirmação. Voltar à sessão do candidato, atualizar a consulta e localizar a notificação da atualização.

Continuidade

Encerrar mostrando o percurso completo: estudar, disponibilizar uma oportunidade, candidatar-se e receber o retorno do processo.

Percurso demonstrado

Aprendizagem e seleção com continuidade

Aprendizagem

A resposta é avaliada, o progresso é registrado e a primeira conclusão válida pode conceder XP.

Oportunidade

Um responsável autorizado publica a vaga e disponibiliza as etapas da seleção.

Candidatura e retorno

O candidato ingressa no processo e acompanha a atualização feita pelo recrutador.

Referências

Documentação, modelos e implementação

Documentos do projeto

01 Documento de visão
02 Atividades do negócio
03 Requisitos do sistema
04 Casos de uso

Modelagem e implementação

Projeto Astah editável revisado
Código-fonte do Stacklyst e modelo de dados Prisma

Orientação do seminário

Sequência e critérios da professora Patrícia, conforme foto da lousa e explicação fornecida pelo grupo.